CM #4
Arbres binaires

TI301 - P2 2025/2026

Nicolas FLASQUE

Prérequis:

Maîtriser les structures et les pointeurs
Savoir representer visuellement des structures de données
Implémentation des piles et des files

Objectifs :

Définir le type `t_node`, utiliser le `type t_node *`
Définir le type `t_tree`
Implémenter les algorithmes classiques d'arbres de manière récursive
Parcours en largeur et en profondeur d'arbres

Organisation :

1 cours (2h)
1 séance d'exercices (2h)
1 session de TP(3h)

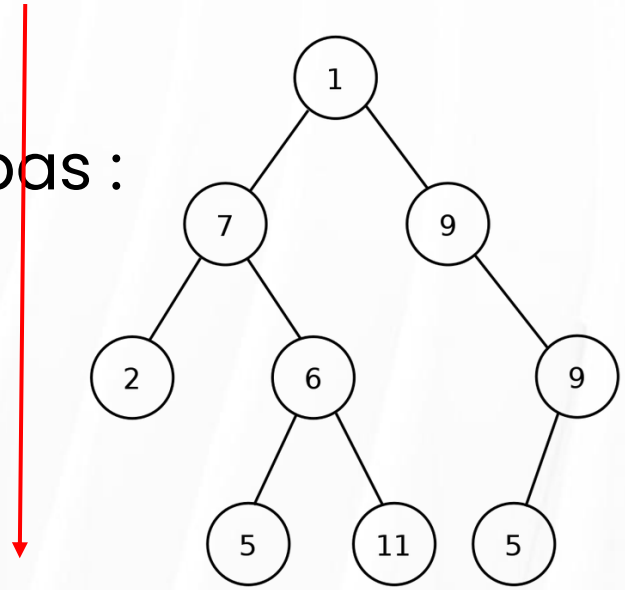
Les structures d'arbres

Assez similaire aux listes :

Dans une liste, chaque élément (`t_cell`) pointe vers la cellule suivante.

Dans un arbre binaire, chaque élément (`t_node`) pointe vers deux autres nœuds.

Les arbres sont représentés de haut en bas :



Le type `t_node`

Un nœud dans un arbre est similaire à une cellule d'une liste, mais stocke **deux** pointeurs.

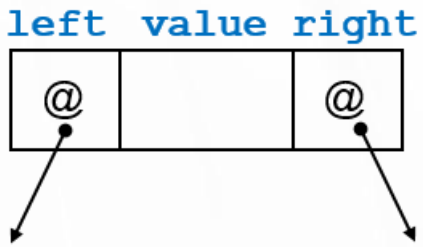
Afin de les distinguer et de faciliter la visualisation, ces pointeurs seront nommés '`left`' et '`right`'.

Ainsi, la définition du type `t_node` :

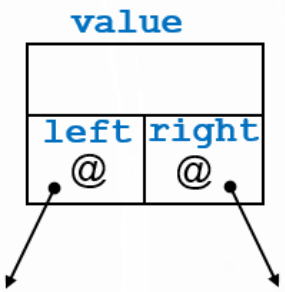
```
typedef struct s_node
{
        struct s_node *left ;
        T value ; // T est un type quelconque : int, float ou autre.
        struct s_node *right ;
} t_node;
```

Représentation d'un nœud

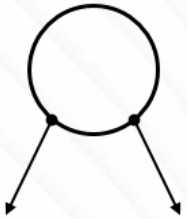
La représentation standard d'un noeud serait :



Nous pouvons utiliser une représentation compacte :



Ou même utiliser ce genre de symbole :



Plus confortable, mais pas de représentation explicite des pointeurs !

Similarités avec les cellules d'une liste

La création d'un nœud se fait via un algorithme à réutiliser:

```
createNode() ;
```

Cet algorithme doit :

1) Prendre comme paramètre : la valeur à stocker dans le noeud ;
2) Retourner un pointeur vers un noeud (type `t_node *`).

Ainsi, son prototype sera :

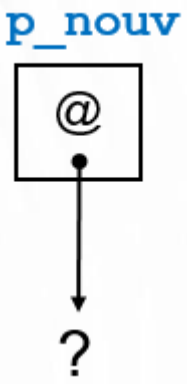
```
t_node *createNode(T val);
```

<u>Rappelez-vous</u>, `T` est un "certain type" : soit `int`, soit `float`, soit tout autre type…

Visualisation de la creation d'un t_node – stockant des 'int'

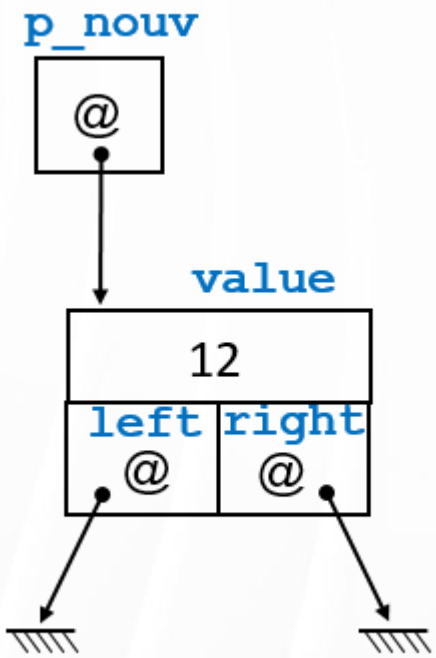
```
t_node *p_nouv;
```

// un pointeur non initialisé



// après appel à createNode()

```
p_nouv = createNode(12);
```



Le type `t_tree`

Une fois de plus, comme pour les listes, un arbre est une structure, contenant un pointeur vers le "premier" nœud de l'arbre.

Selon la convention de dénomination habituelle, le "premier" nœud d'un arbre est appelé **"racine"** de l'arbre.

Ainsi, la définition du type `t_tree` :

```
typedef struct s_tree
{
        t_node *root ;
} t_tree ;
```

Pour comparaison / analogie :

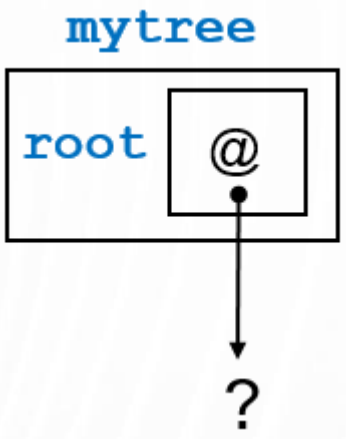
```
typedef struct s_std_list
{
        t_cell *head;
} t_std_list;
```

Représentation

Soit la définition de la variable suivante :

```
t_tree mytree;
```

Alors sa représentation est :

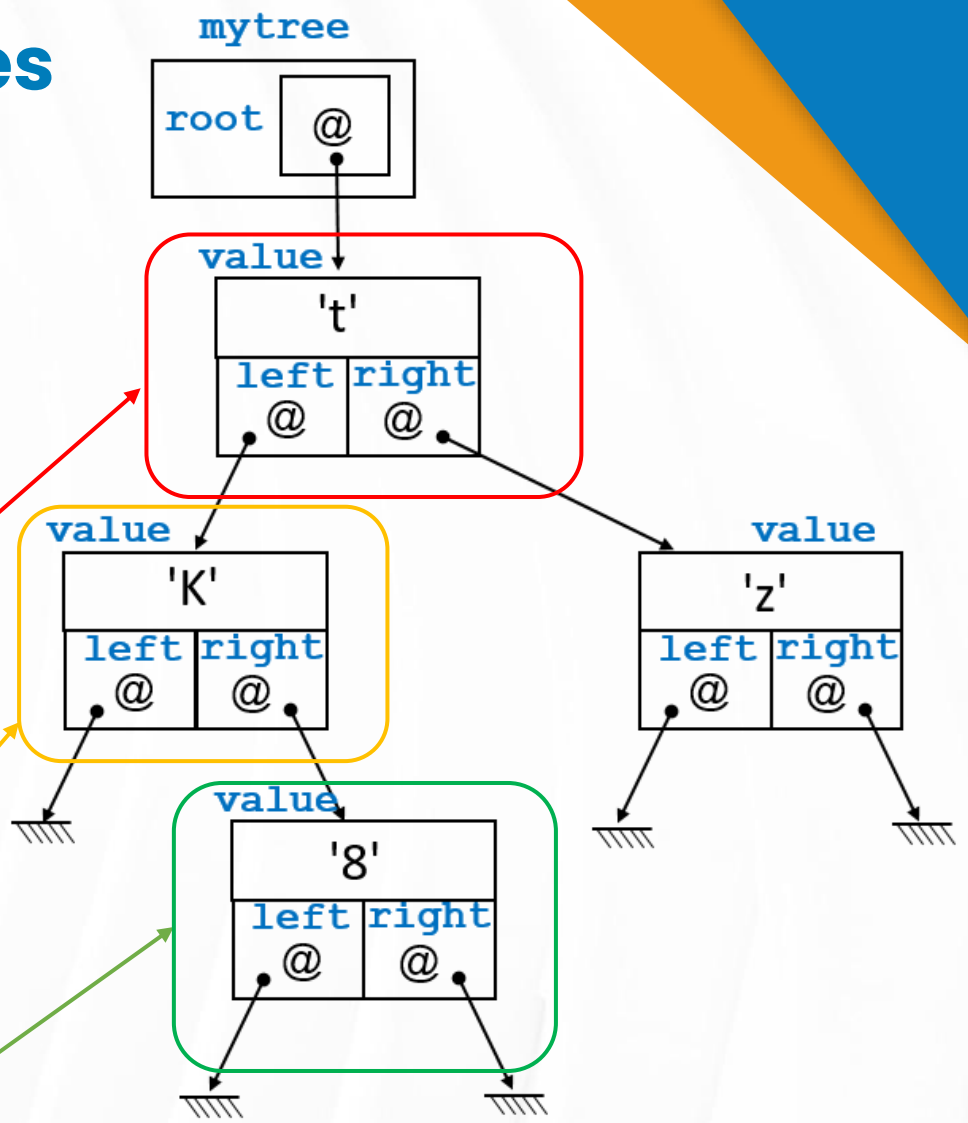


Où `mytree.root` est un pointeur non initialisé.

Représentation des arbres non vides

Supposons que le type de valeur stockée dans les noeuds est `'char'`. (chaque noeud stocke une valeur `'char'`).

Voici un exemple de la façon dont un tel arbre est représenté

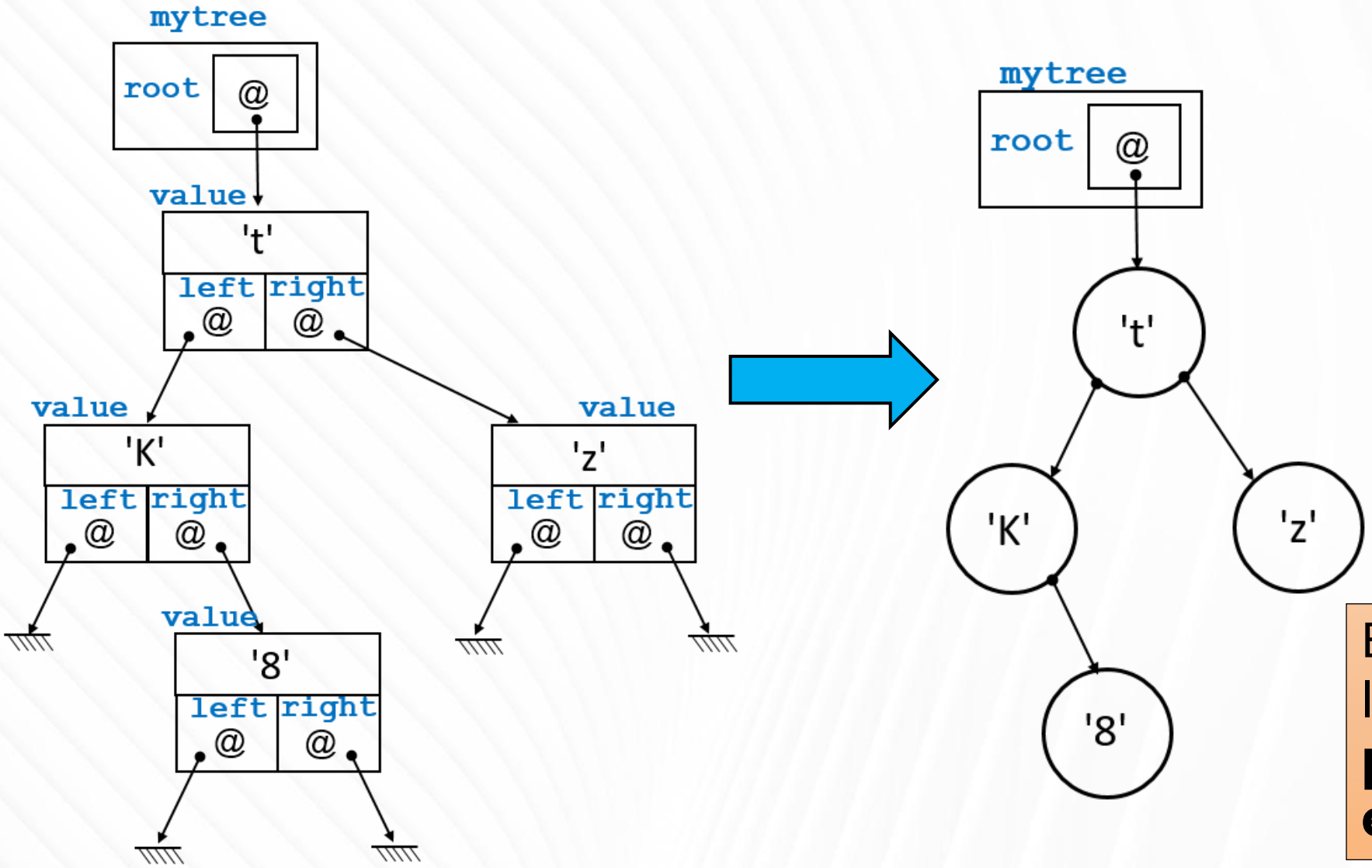


Certains nœuds ont des **"enfants"** `left` et `right`.

Certains nœuds ont un **"enfant"** `left` ou `right`, l'autre étant `NULL`.

Certains nœuds ont à la fois les **"enfants"** `left` et `right` à `NULL` : ces nœuds "terminaux" sont appelés "**feuilles**" de l'arbre.

Représentation alternative



Beaucoup plus facile à lire, mais **aucun pointeur n'est explicitement illustré** !

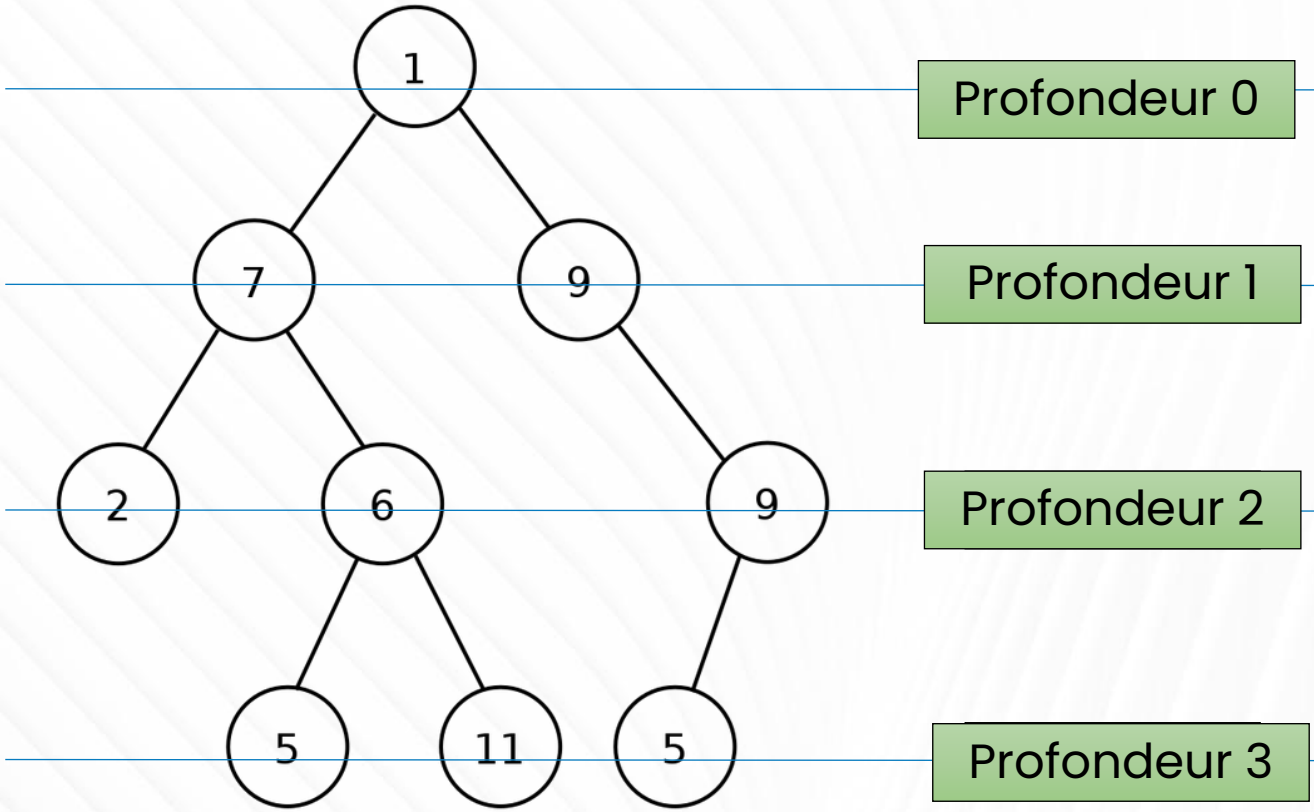
Représentation utilisée dans les TPs

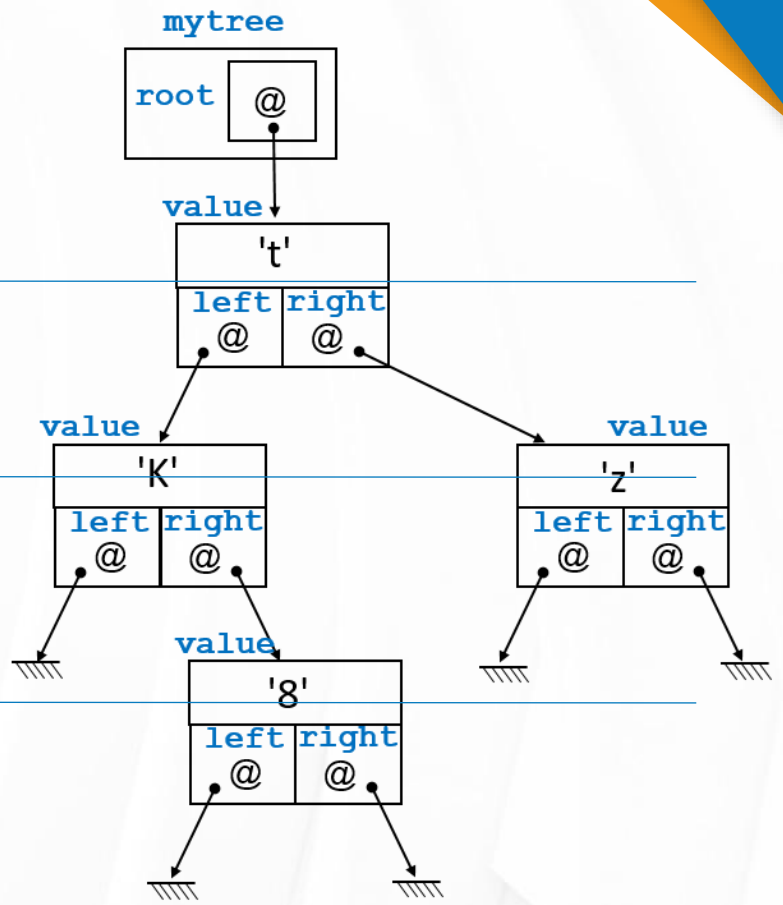
Une fonction `displayTree()` sera disponibles pour vous lors des TPs à venir sur les arbres. Un arbre sera affiché comme tel :

```
                                  [root@]
                                     |
                                     v
                                  [-08]
                       [L@]--------+-------[R@]
                        |                   |
                        v                   v
                      [-07]               [006]
            [L@]--------+--[R@]      [L@]---+-------[R@]
             |              |         |              |
             v              v         v              v
           [-05]          NULL      NULL           [010]
      [L@]---+--[R@]                           [L@]---+-----------------[R@]
       |         |                              |                       |
       v         v                              v                       v
     NULL      NULL                           NULL                    [008]
                                                          [L@]--------+--[R@]
                                                           |              |
                                                           v              v
                                                         [005]          NULL
                                                   [L@]---+--[R@]
                                                    |         |
                                                    v         v
                                                  NULL      NULL
```

Définitions standards

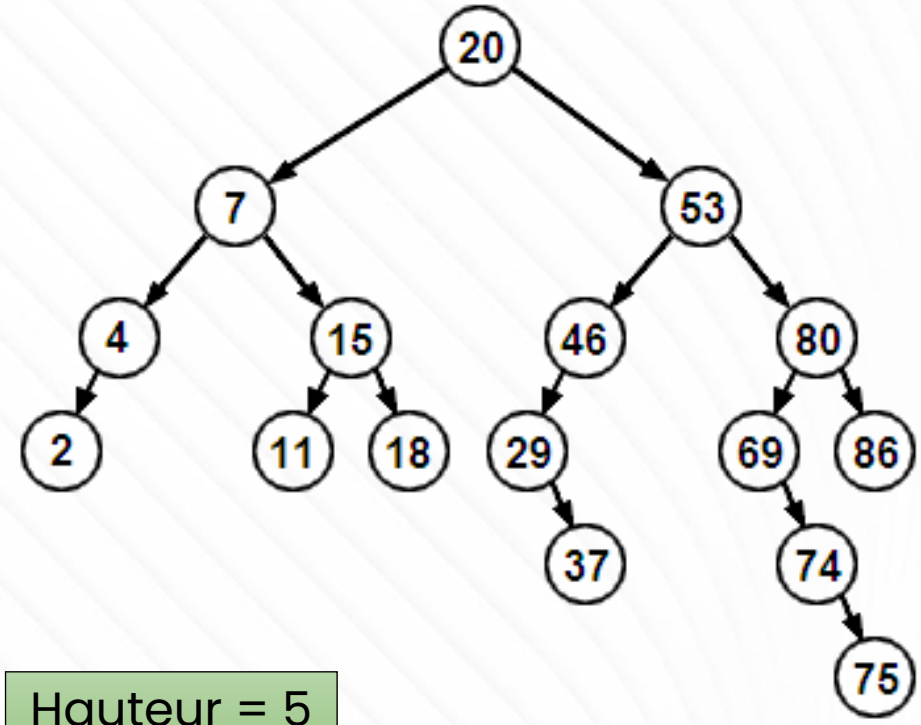
La **"profondeur**" d'un nœud est sa "distance" au nœud racine : chaque "rangée" de l'arbre est au même profondeur.





Définitions standard

La "**hauteur**" d'un arbre est égale à la profondeur maximale parmi tous ses nœuds.



Hauteur = 5

```
                    [root@]
                       |
                       v
                    [-08]
            [L@]--------+-------[R@]
              |                   |
              v                   v
            [-07]               [006]
      [L@]--------+--[R@]    [L@]---+-------[R@]
        |             |        |              |
        v             v        v              v
      [-05]         NULL     NULL           [010]
  [L@]---+--[R@]                        [L@]---+-----------------[R@]
    |         |                           |                       |
    v         v                           v                       v
  NULL      NULL                        NULL                    [008]
                                                      [L@]--------+--[R@]
                                                        |              |
                                                        v              v
                                                      [005]          NULL
                                                  [L@]---+--[R@]
                                                    |         |
                                                    v         v
                                                  NULL      NULL
```

Hauteur = 4

De : https://algo.developpez.com/faq/ ? page=Structures-arborescentes

Récursivité et arbres

Pour les listes : la récursivité et l'itération peuvent toutes deux être utilisées

Pour les arbres : la récursivité est un moyen "naturel" d'exprimer les opérations classiques

BONNE NOUVELLE

Nous devons utiliser la récursivité pour traiter les arbres **de manière simple et efficace**

BONNE NOUVELLE ?

Vous devez maîtriser la récursivité pour traiter des arbres **de manière simple et efficace**

Définition récursive d'un arbre - sous-arbres

Soit `mytree` une variable de type `t_tree`.

Question : Quel est le type de `mytree.root` ?

`t_node *`

Indice :

```
typedef struct s_tree
{
        t_node *root ;
} t_tree ;
```

Question : Quel est le type des pointeurs `left` et `right` d'un noeud ?

Indice :

```
typedef struct s_node
{
        struct s_node *left ;
        T value ; // T est un type quelconque : int, float ou autre.
        struct s_node *right ;
} t_node;
```

`struct s_node *`, aka `t_node *`

Définition récursive d'un arbre - sous-arbres

Soit `mytree` une variable de type `t_tree`.
Soit `temp` une variable de type `t_node *`

Alors : `mytree.root` et `temp` sont tous deux de type `t_node *`

Cela permet la récursivité, tout comme pour les listes

Pour écrire une fonction récursive pour les arbres :

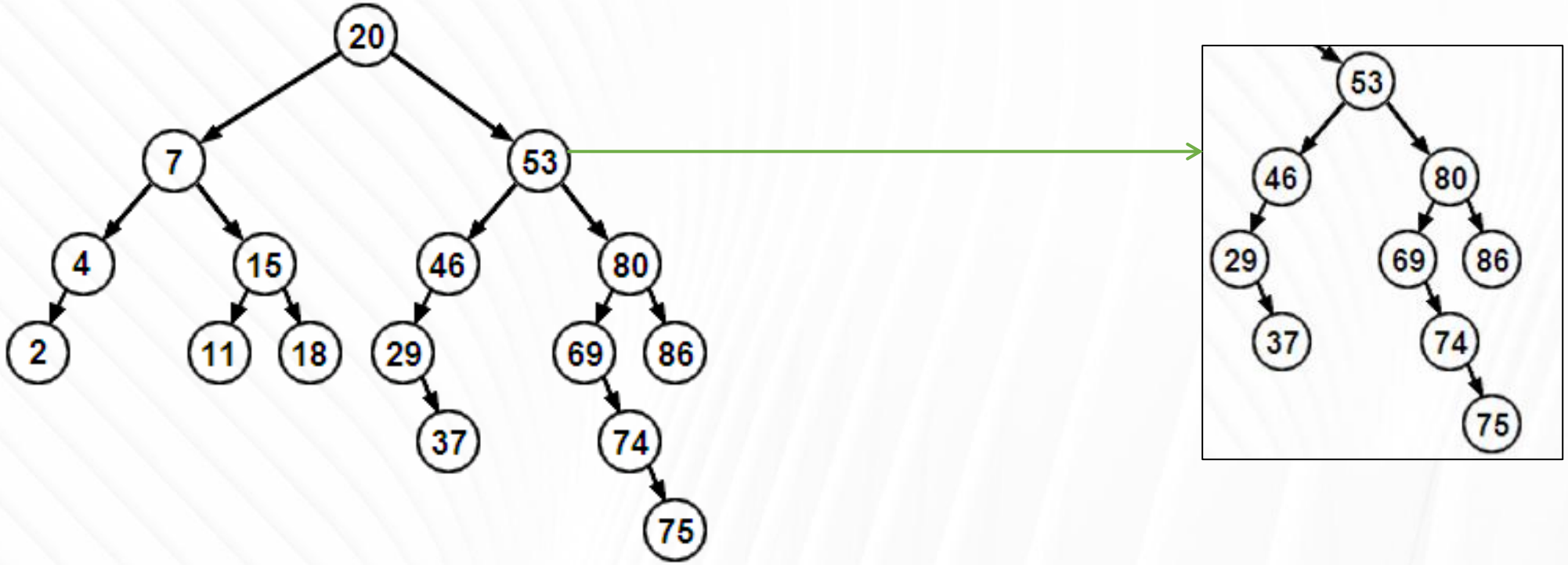
1) Ecrire une fonction récursive pour le type `t_node *`

2) Pour `mytree`, provoquer le premier appel récursif avec `mytree.root`

Relation entre t_node et t_tree

Dans la suite du cours, on associera souvent un noeud `t_node`, ou pointé par un `t_node *`, avec l'arbre dont la racine est ce noeud

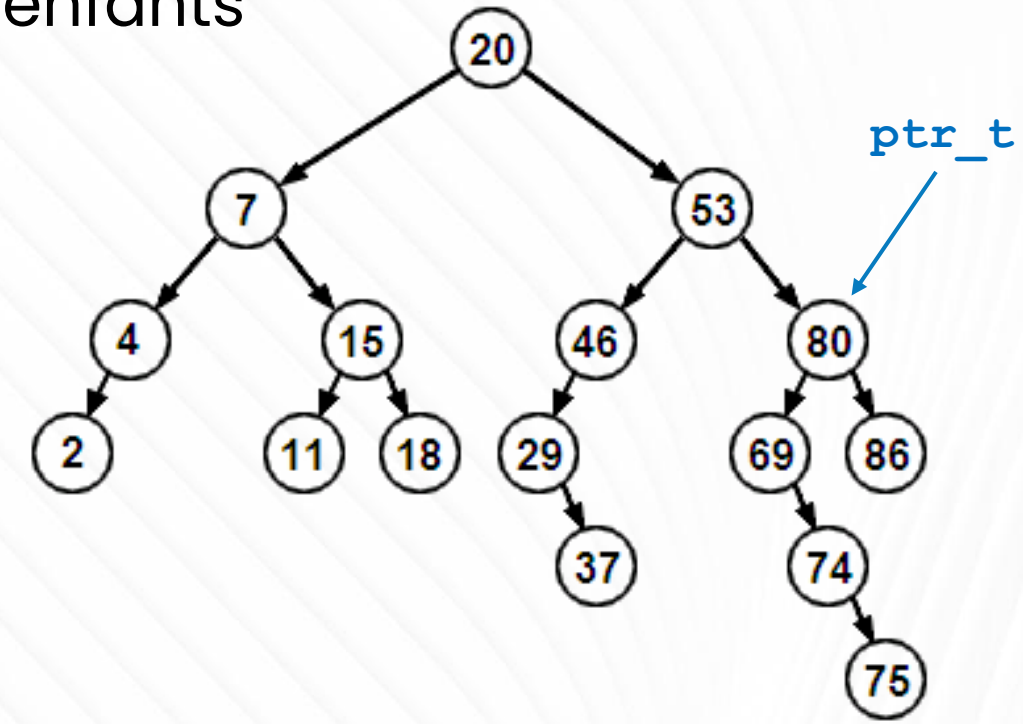
Illustration : on associera le noeud '53' avec tout le 'sous arbre'



Relation entre t_node * et t_tree

Dans la suite du cours, on associera souvent un noeud `t_node`, ou pointé par un `t_node *`, avec l'arbre dont la racine est ce noeud

Illustration : on associera le pointeur '`ptr_t`' avec un noeud et tous ses enfants



Après tout, un arbre est bien repéré par un `t_node *root`

Un exemple : la fonction height()

Comment calculer la hauteur d'un arbre (à partir de `root`) ?

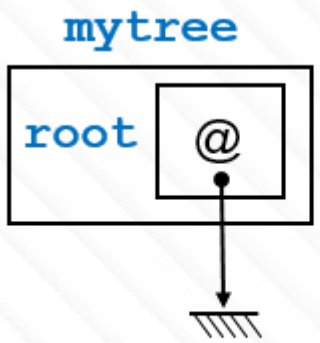
Si l'arbre est vide : sa hauteur est de -1.

Si l'arbre n'est pas vide, sa hauteur est : 1 + le maximum entre
- la hauteur de son "sous-arbre" gauche
- la hauteur de son "sous-arbre" droit

Étape 1) calculer la hauteur d'un `t_node *`

Étape 2) calculer la hauteur de l'arbre à partir de son `root`

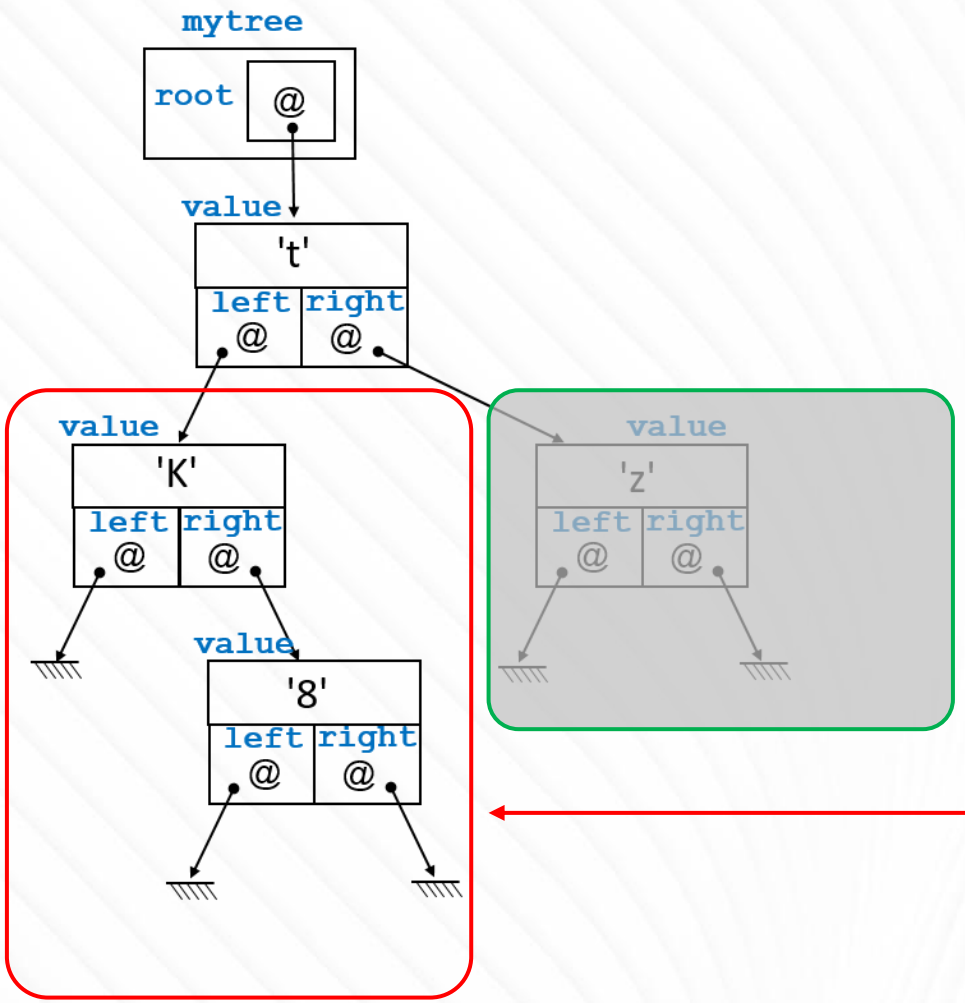
Exemples



La hauteur de `mytree` est la hauteur de son champ `root`.

Ici, la hauteur est de -1, car `mytree.root == NULL`

Exemples



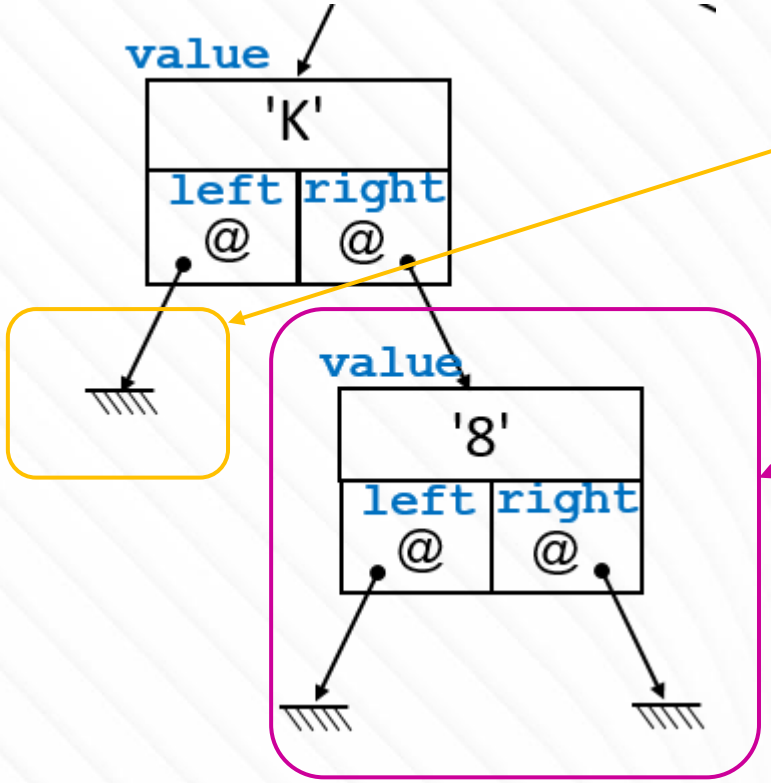
La hauteur de "ce qui reste à gauche" est la hauteur du champ `root->left` :

Si c'est `NULL`, c'est 0.
Si non : 1+max('ce qui reste à gauche', 'ce qui reste à droite') ;

Exemples

Si le pointeur vaut `NULL` : la hauteur vaut 0

Si non : 1+max('ce qui reste à gauche', 'ce qui reste à droite') ;



Algorithme récursif height()

1) Étape 1 : hauteur d'un noeud `t_node *`

Si le pointeur est `NULL` : sa hauteur est de -1.

Sinon : c'est 1+max('ce qui reste à gauche', 'ce qui reste à droite')

Écrivons la fonction `nodeHeight()` :

Question : Que doit retourner cette fonction ?

Une valeur `int`

Question : le pointeur `root` est-il modifié lors du calcul de sa hauteur ?

Non

fonction récursive Height() : prototype

D'où le prototype :

```
int nodeHeight(t_node *);
```

Puis la définition

```
int nodeHeight(t_node *p_n);
{
        int height;
        // c'est récursif, donc ...
        return height;
}
```

La suite :

D'où le prototype

```
int nodeHeight(t_node *p_node);
```

puis l'algorithme…

```
int nodeHeight(t_node *p_n);
{
        int height;
        // c'est récursif, donc ...
        return height;
}
```

```
// Un node 'NULL' est de hauteur -1
if (p_node == NULL)
    height = -1;
// sinon on appellee nodeHeight() pour chaque noeud fils
else
{
    height = 1+max(nodeHeight(p_node->left),nodeHeight(p_node->right));
}
```

Compter le nombre de nœuds dans un arbre binaire?

Au moyen d'un algorithme récursif `nodeCount()`

À voir en TD/TP

Créer un arbre binaire aléatoire

Soit `mytree` un `t_tree` stockant des `char`.

On utilisera la fonction `emptyTree()` pour l'initialiser :

```
t_tree emptyTree()
{
   t_tree atree;
   atree.root = NULL;
   return atree;
}
```

Créer un arbre binaire aléatoire

Objectif : créer un nouveau nœud et trouver un endroit dans l'arbre où l'insérer

Cet endroit sera choisi aléatoirement :

<u>À CHAQUE ÉTAPE:</u>

Choisir aléatoirement entre 'left' et 'right'

S'il n'y a pas encore de nœud à cet endroit,
Y insérer le nouveau nœud

Sinon, y aller et …

Faîtes attention au cas de l'arbre vide

La fonction `addRandomNode()`

Prérequis :

Prend en paramètre une valeur de type `char`, et l'arbre dans lequel nous voudrions ajouter le noeud

Est-ce que cette fonction est susceptible de modifier l'arbre? ⇔ (est-ce que le champ `'root'` peut-être modifié?)

Oui! Si l'arbre est vide alors le champ `'root'` sera modifié.

Donc le prototype sera :

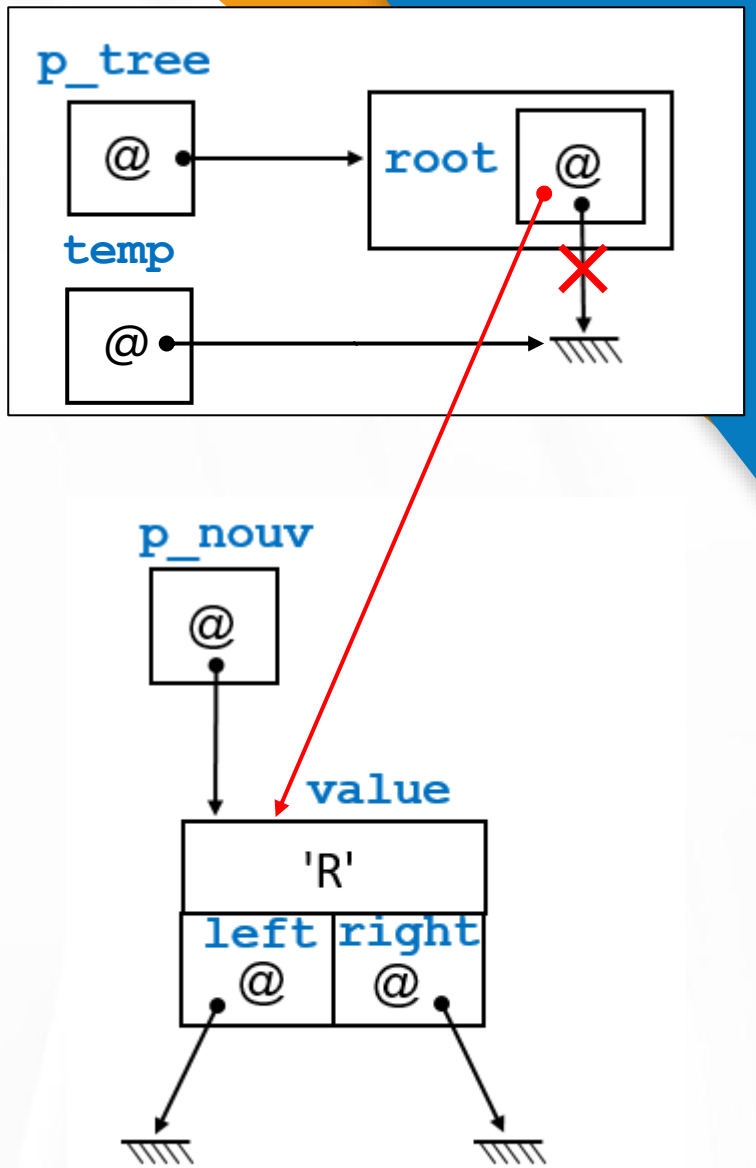
```
void addRandomNode(t_tree *,char);
```

Visualisation

```
void addRandomNode(t_tree *p_tree,char val)
{
    t_node *p_nouv;
    t_node *temp;

    p_nouv = createNode(val);
    temp = p_tree->root;

    // test si arbre vide
    if (p_tree->root == NULL)
        p-tree->root = p_nouv;
```



Si l'arbre n'est pas vide

`temp` sera notre curseur pour parcourir l'arbre

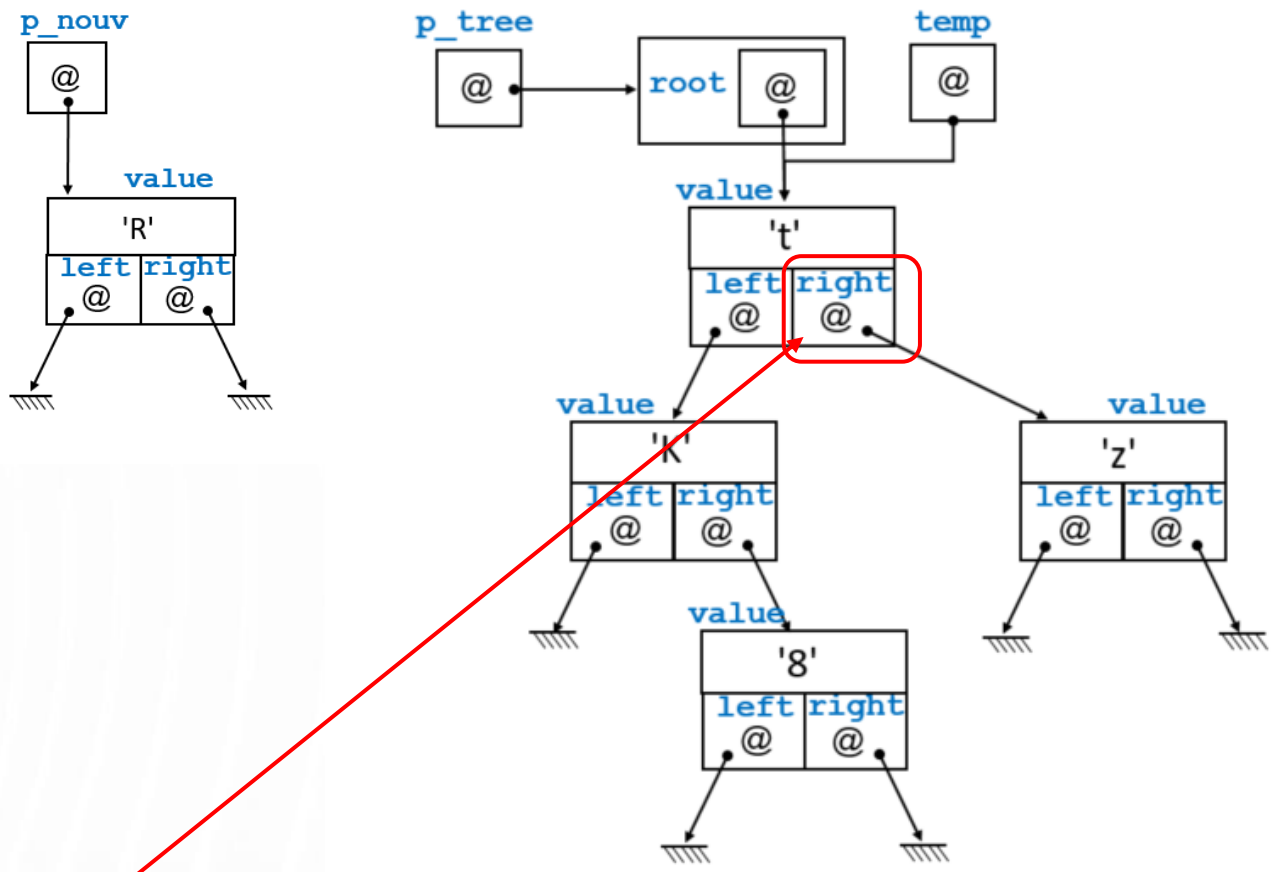
Choisir aléatoirement entre:
0 (essayer à gauche) et 1 (essayer à droite)

Exemple : On tire 1 donc on essaie à droite

Si l'endroit est libre (pas de nœud) on peut insérer `p_nouv` à droite de `temp` ?

Quelle condition vérifie que l'endroit est libre ?

```
Si(temp->right == NULL)alors,
l'emplacement est disponible
```



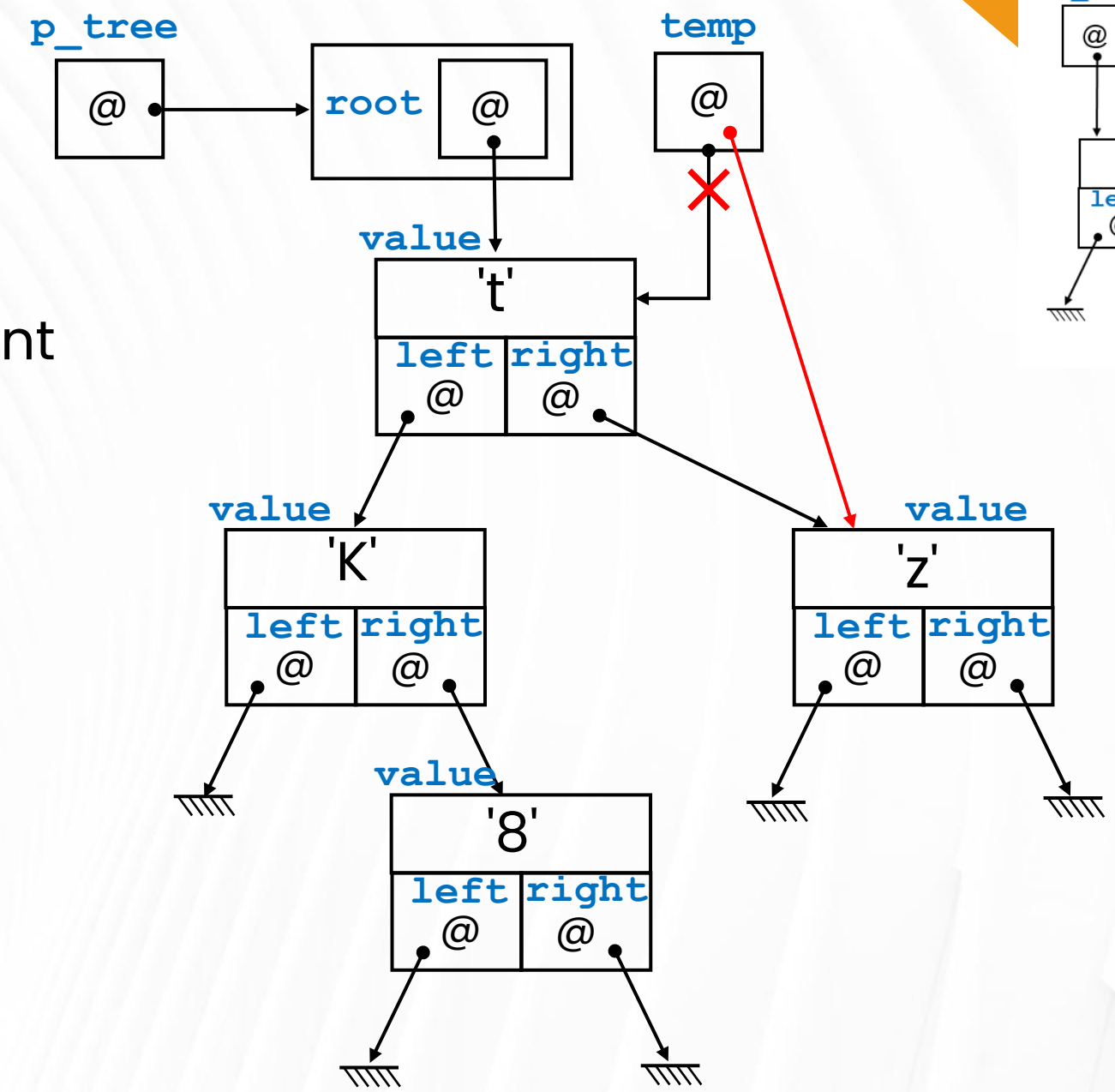

```
Si(temp->right != NULL) alors,
l'emplacement n'est pas disponible
```

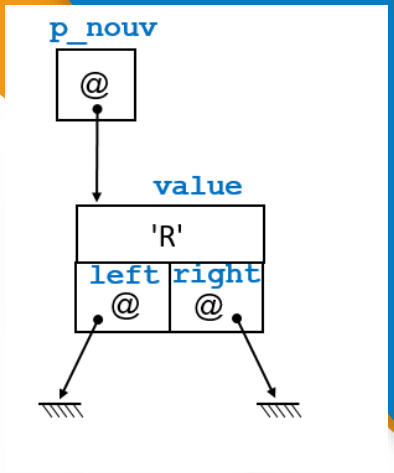
Allons à droite

Ici, ce n'est pas disponible, alors, nous allons à cet emplacement et relançons le tirage

Se déplacer à droite signifie

```
temp = temp->right;
```





Suite

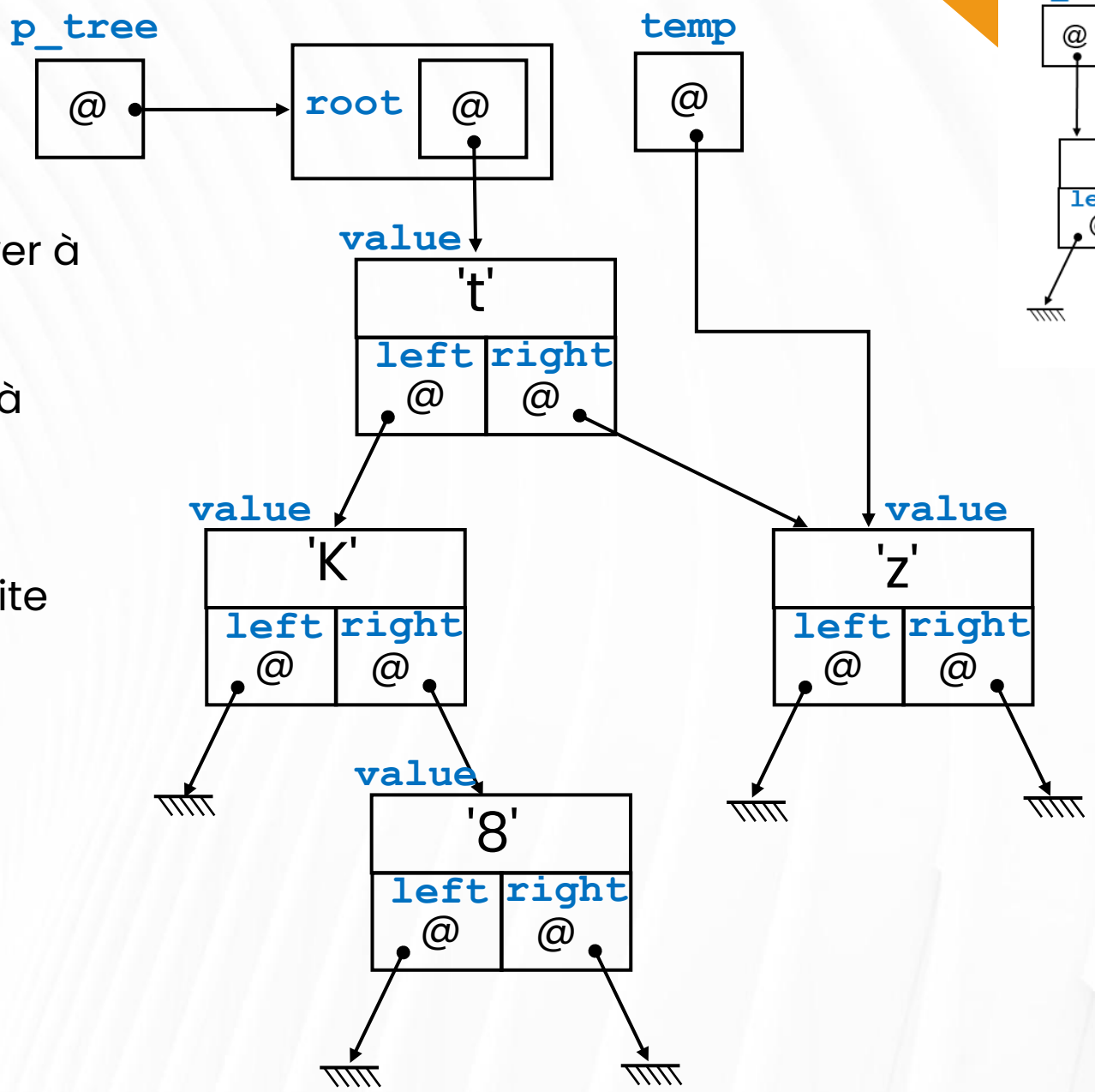
Choisir aléatoirement entre:
0 (essayer à gauche) et 1 (essayer à droite)

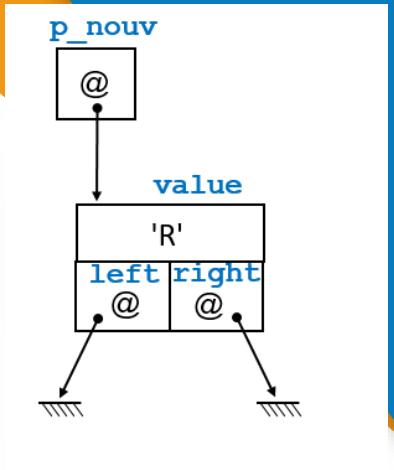
Exemple : On tire 1 donc essayer à droite
(encore !)

Est-ce que l'emplacement à droite de `temp` est libre pour insérer `p_nouv` ?

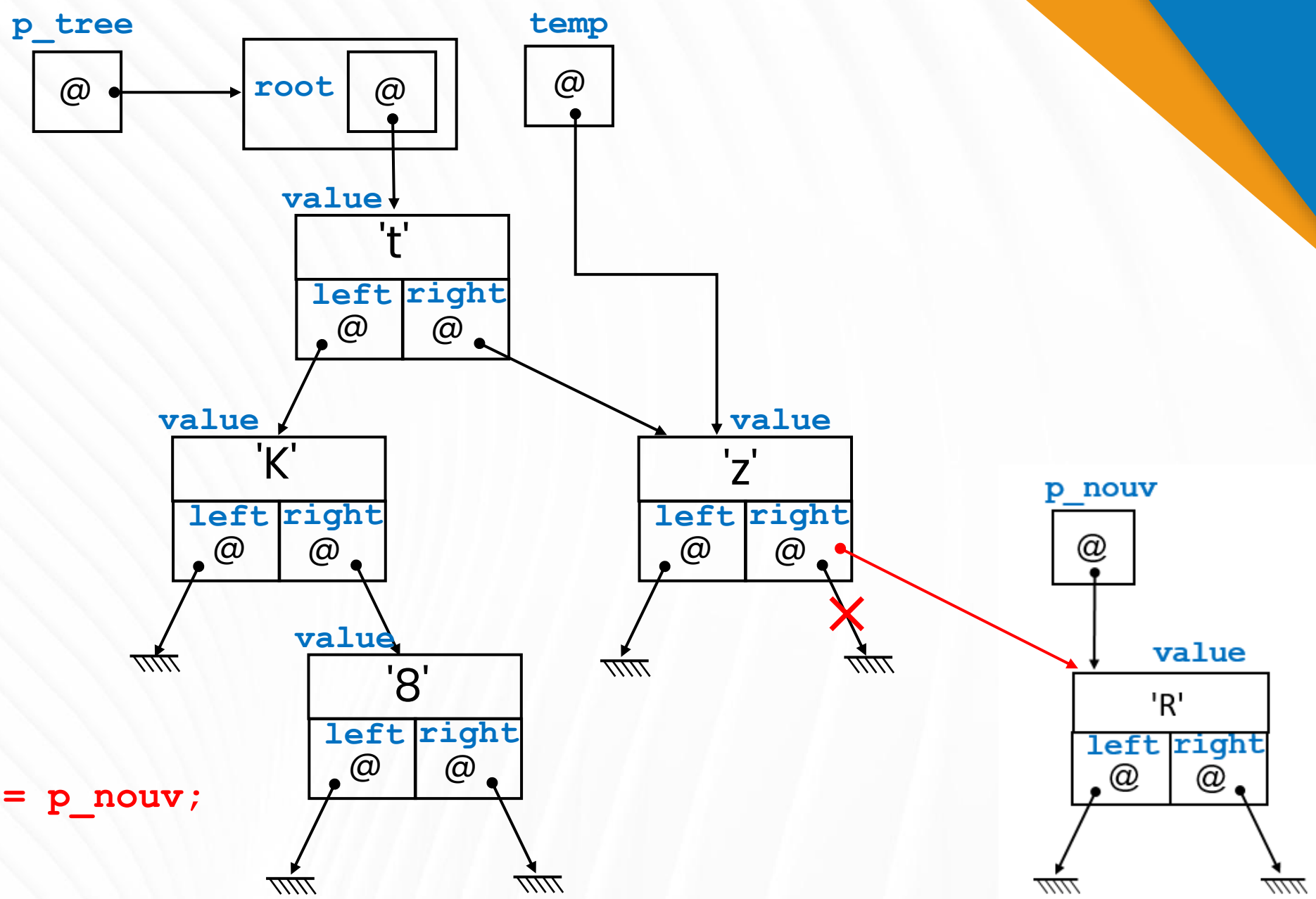
Oui car `temp->right` vaut `NULL`

Alors, insérer `p_nouv` à droite de temp.





Dernière étape




```
temp->right = p_nouv;
```

`addRandomNode()`

La suite en TD/TP

Parcourir un arbre (comparé aux listes)

Il existe de nombreuses manières différentes de " visiter " ou de parcourir tous les nœuds d'un arbre.

Dans une liste (std) , il n'y a pas d'autre choix que d'aller 'en avant', en utilisant le pointeur `next`.

Dans un arbre, il y a un choix à faire entre 'gauche' et 'droite' (gauche seulement ? gauche d'abord ? droite seulement ? droite d'abord ?).

Nous verrons comment organiser la récursivité pour ordonner le parcours.

Les **files** et les **piles** nous seront également utiles pour un parcours itératif des arbres.

Schéma général de récursivité

Soit `processNode()` une function récursive qui effectue une opération sur les noeuds :

```
type processNode(t_node *, ...)// peut-être d'autres paramètres

if (p_n != NULL)// pas toujours necessaire mais peut vous sauver la vie
{
   // bloc d'instructions (1)
   if (p_n->left != NULL)
      appel récursif : ... processNode(p_n->left, ...)
   // bloc d'instructions (2)
   if (p_n->right != NULL)
       appel récursif :... processNode(p_n->right, ...)
   // bloc d'instructions (3)
   return ...
}
```

Parcours en profondeur

La récursivité précédente applique un parcours dit "en profondeur" d'un arbre binaire.

Les positions des blocs d'instructions (par rapport aux appels récursifs), indiquent 3 principaux types de parcours en profondeur:

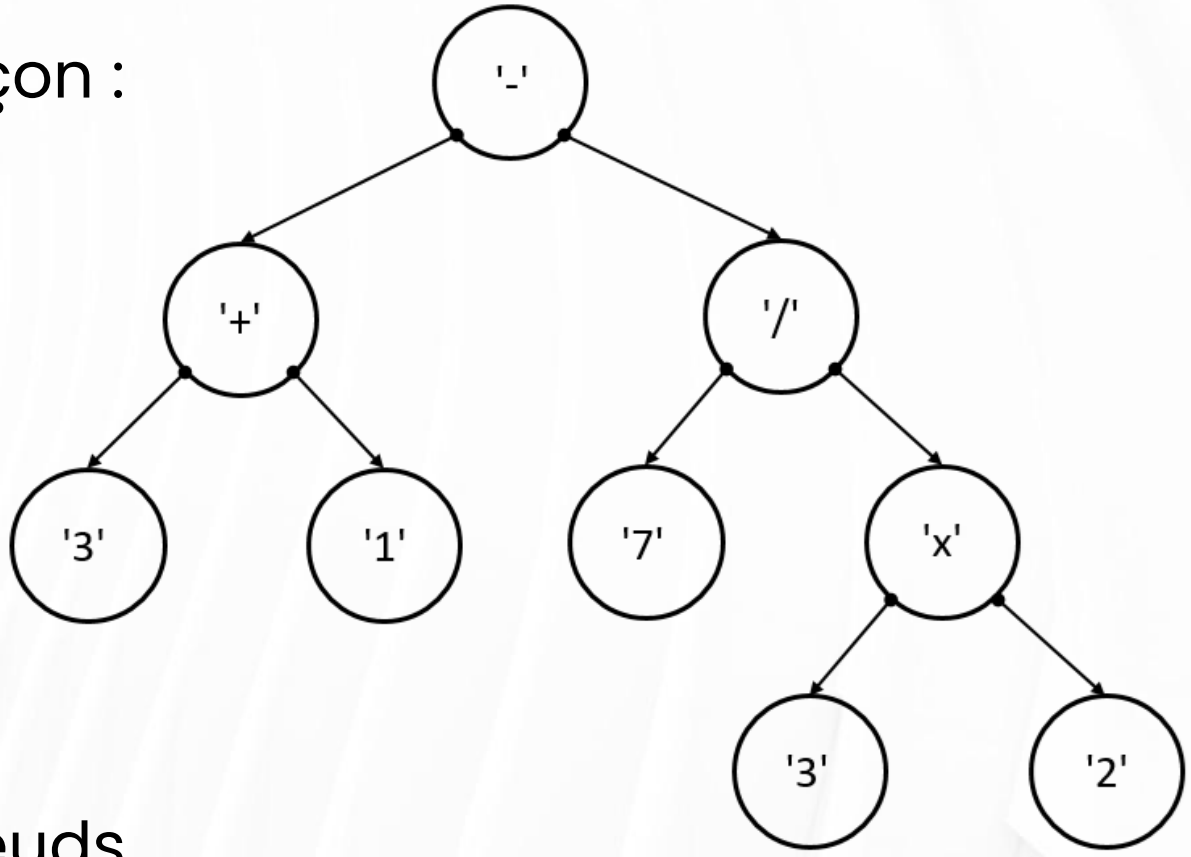
1. Parcours dit 'préfixe'
2. Parcours dit 'infixe'
3. Parcours dit 'postfixe'

Exemples à venir !

Illustration

Soit `mytree` un arbre binaire stockant des valeurs de type `char`.

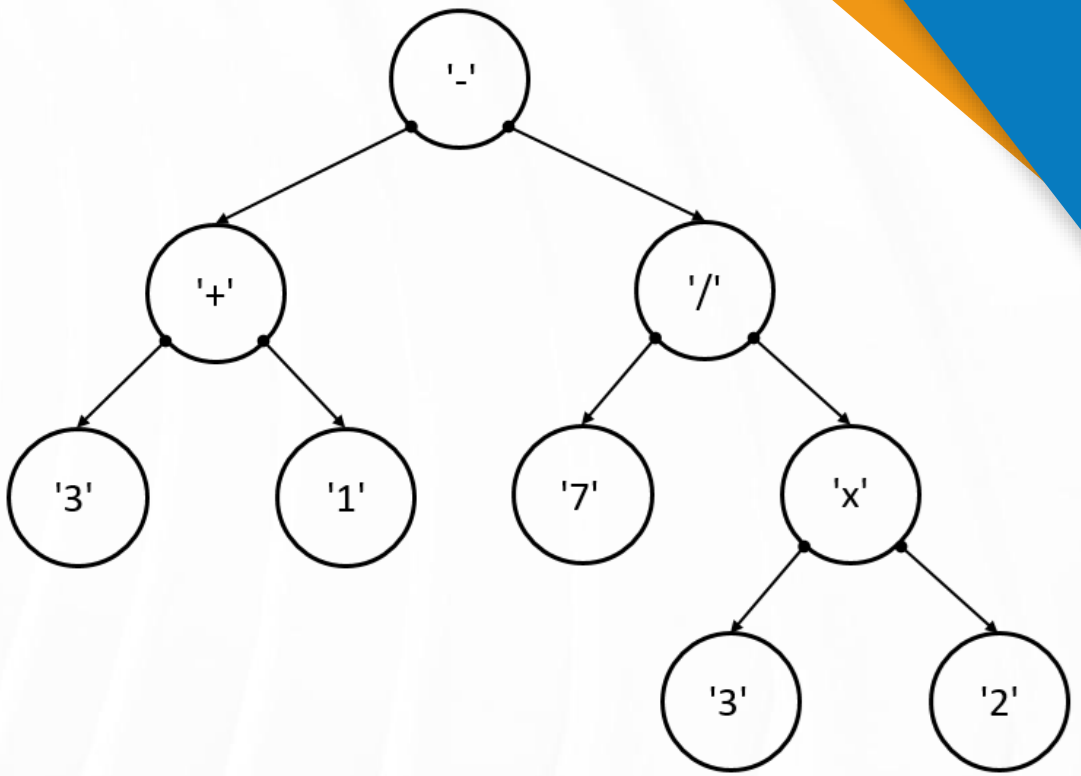
`mytree` est initialisé de cette façon :



Exemples de parcours
pour afficher les valeurs des noeuds

Parcours préfixe

```
void prefixTraversal(t_node *p_n)
{
    if (pn != NULL)
    {
        printf("%c\n",pn->valeur);
        // appel récursif pour pn->left
        prefixTraversal(pn->left);
        // appel récursif pour pn->right
        prefixTraversal(pn->right);
    }
    return;
}
```



Code couleur :

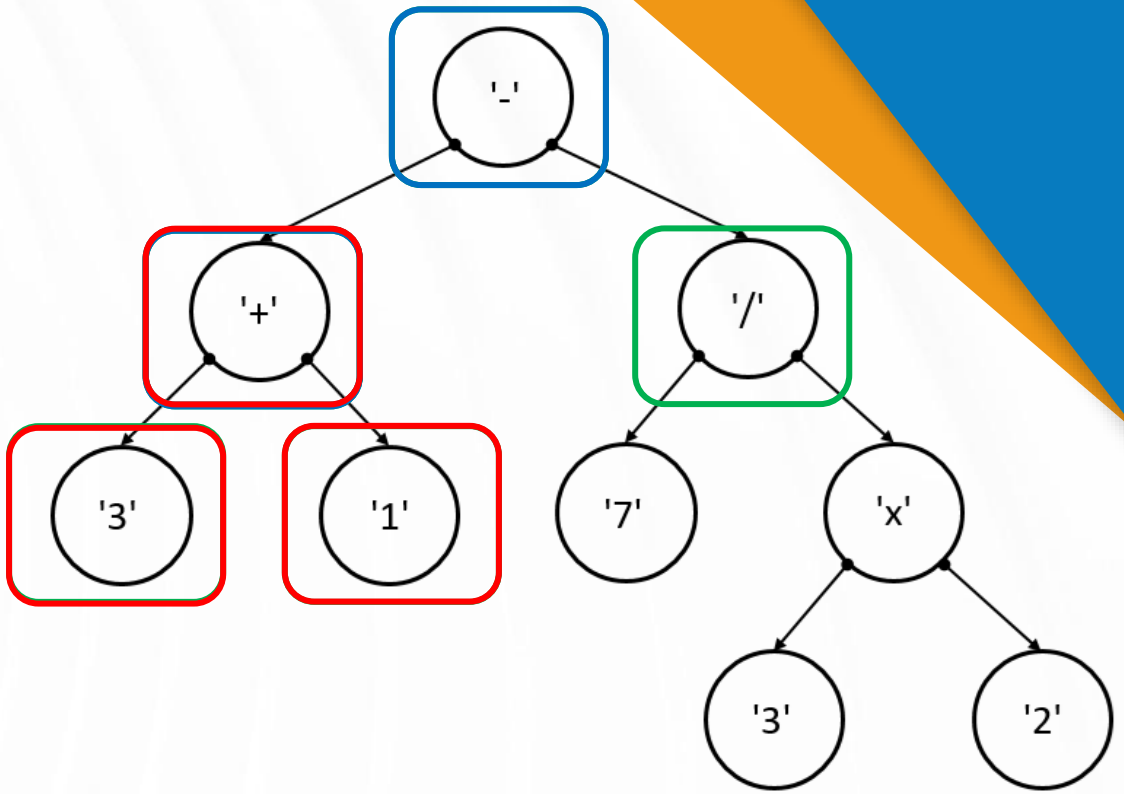
Fonction active entourée en vert
Fonction en attente entourée en bleu
Fonction terminée entourée en rouge

```
Algorithme prefixTraversal pour un pointeur
p_n

afficher(pn->valeur)
appel récursif pour pn->left
appel récursif pour pn->right
```

Sorties

```
-
+
3
1
/
7
x
3
2
```

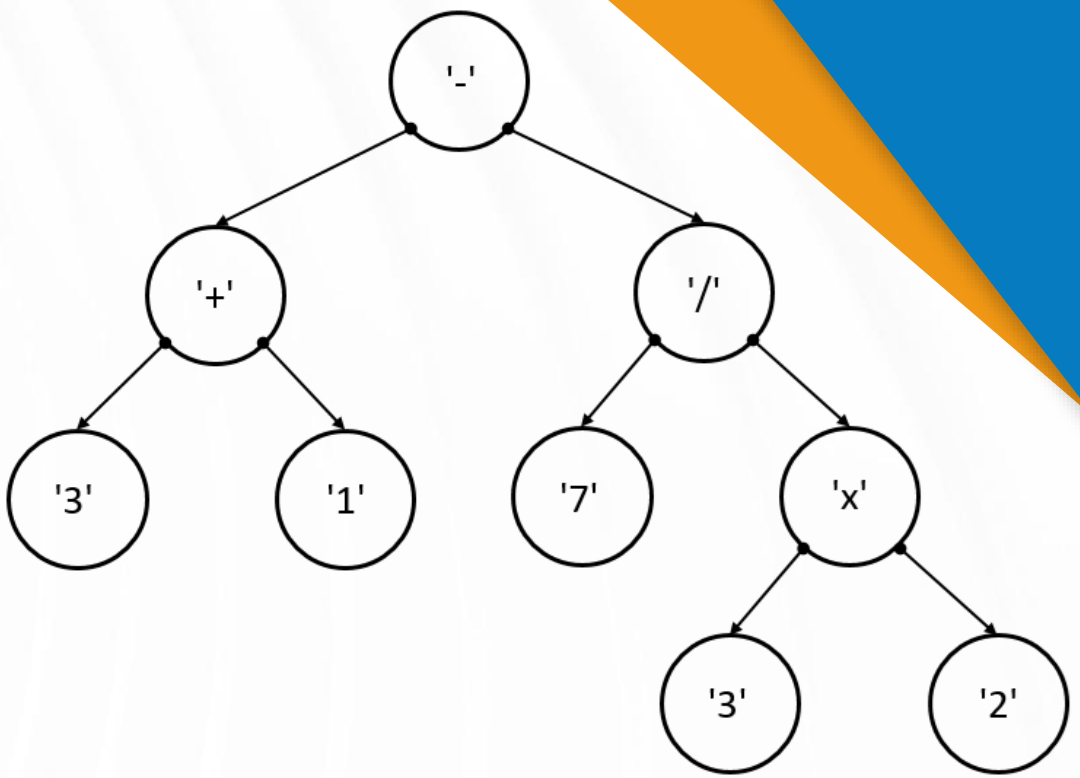


Parcours préfixe

Les anciennes calculatrices PN (Polish Notation) utilisent ce type de notation pour entrer les opérations !

Parcours postfixe

```
void postfixTraversal(t_node *p_n)
{
    if (pn != NULL)
    {
        // appel récursif pour pn->left
        postfixTraversal(pn->left);
        // appel récursif pour pn->right
        postfixTraversal(pn->right);
        printf("%c\n",pn->valeur);
    }
    return;
}
```

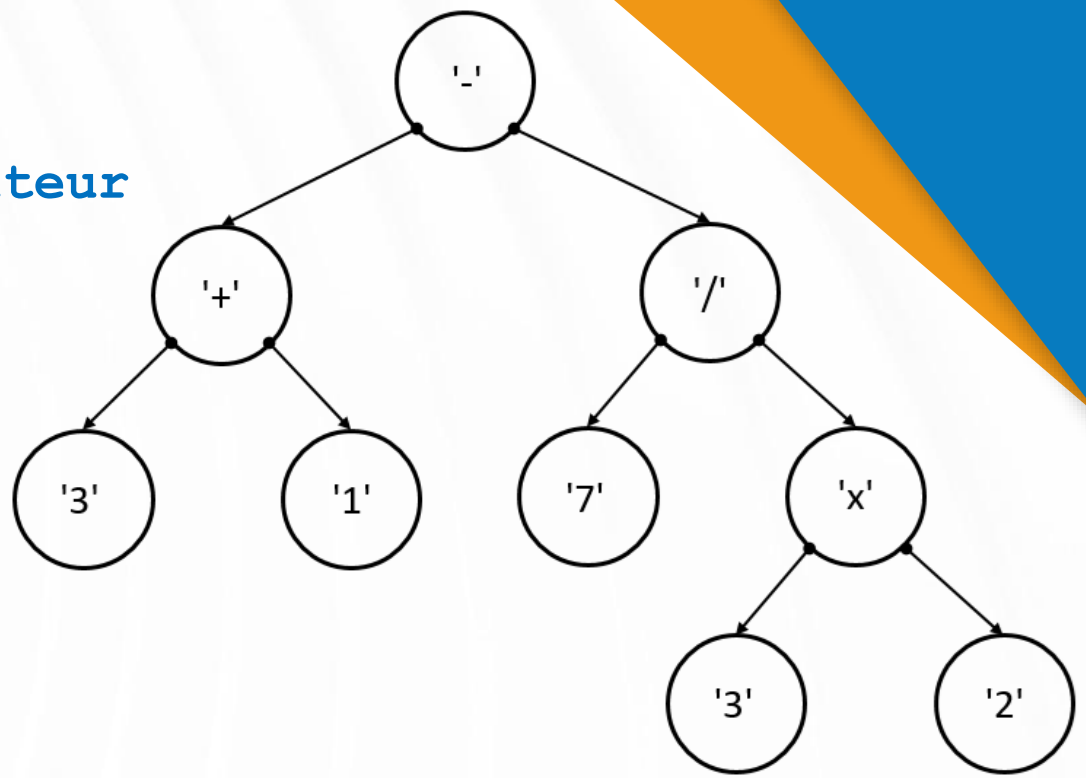


Trouvons ensemble les sorties :

Parcours postfixe

```
Algorithme postfixTraversal pour un pointeur
p_n

appel récursif pour pn->left
appel récursif pour pn->right
afficher(pn-> valeur)
```



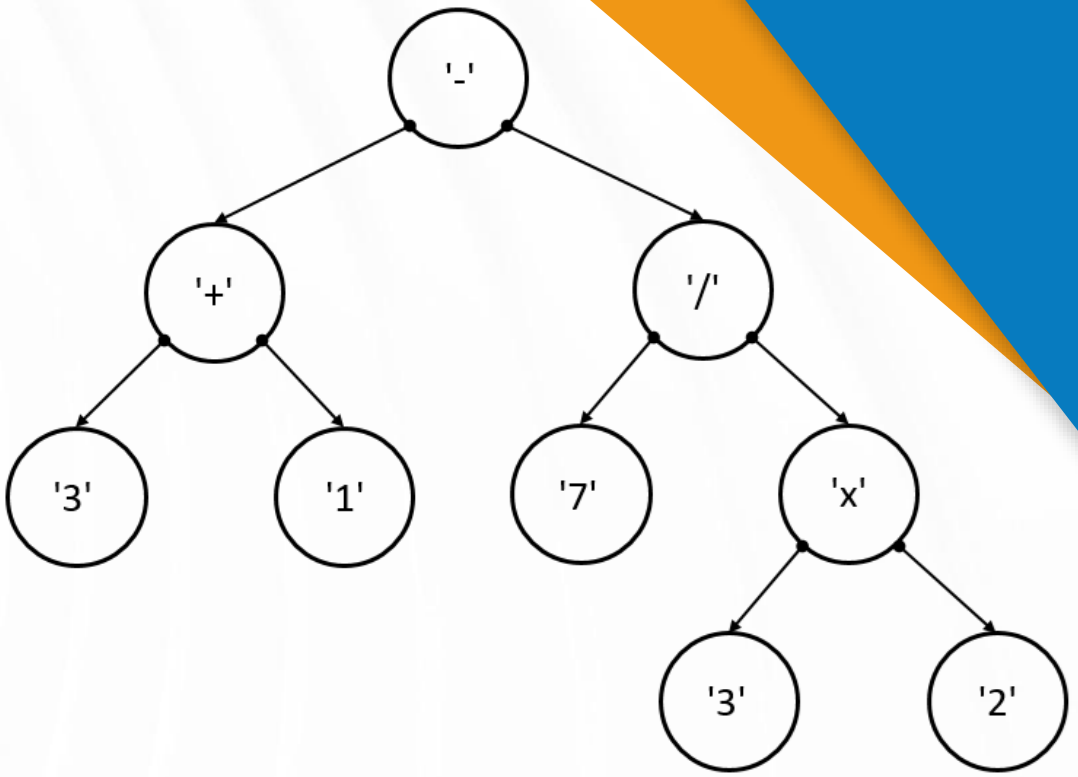
Sorties

```
3
1
+
7
3
2
x
/
-
```

parcours infixe

```
void infixTraversal(t_node *p_n)
{
    if (pn != NULL)
    {
        // appel récursif pour pn->left
        infixTraversal(pn->left);

        printf("%c\n",pn->valeur);
        // appel récursif pour pn->right
        infixTraversal(pn->right);
    }
    return;
}
```

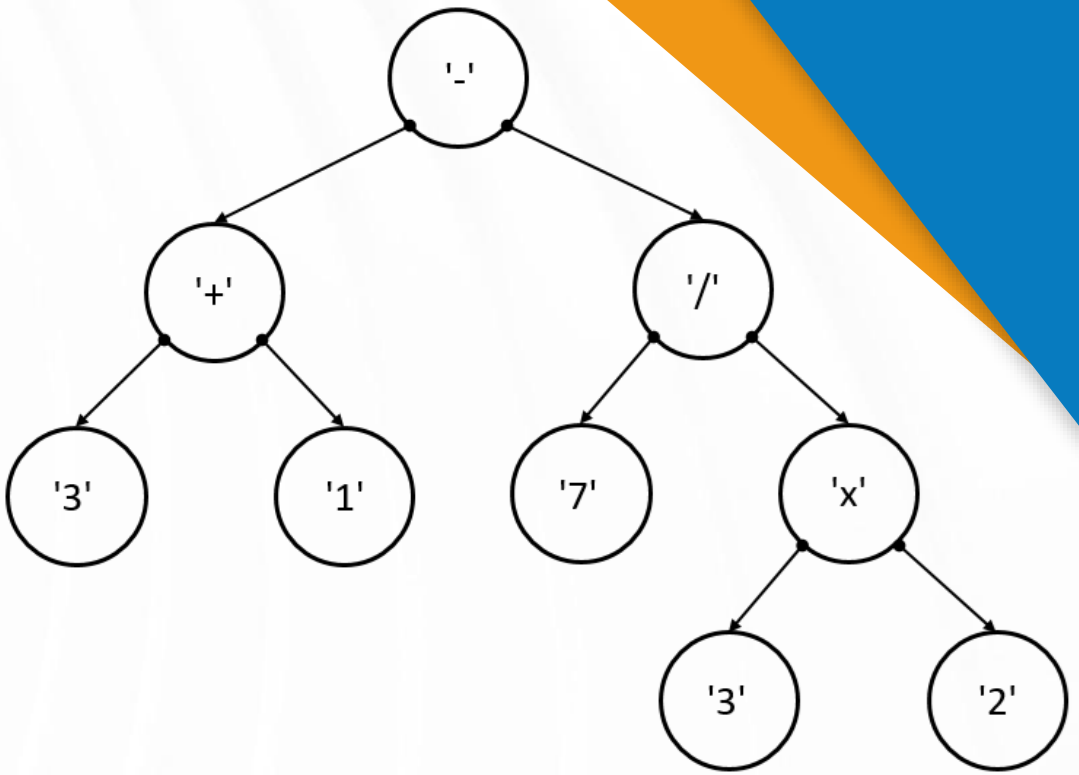


Trouvons ensemble les sorties :

parcours infixe

```
infixTraversal pour un pointeur p_n

appel récursif pour pn->left
Afficher(pn->valeur)
appel récursif pour pn->right
```



Trouvons ensemble les sorties :

Sorties
3
+
1
-
7
/
3
x
2

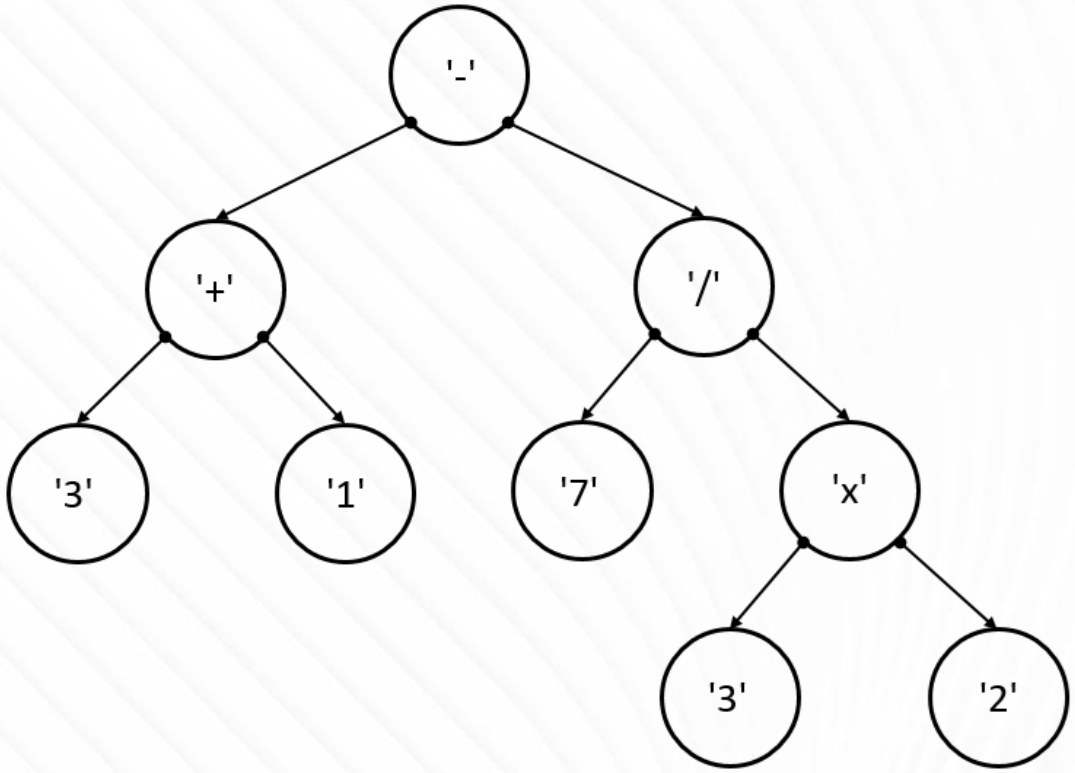
Sorties
3+1-7/3x2

Manipulation des parenthèses et des priorités : TD/TP

Catégories d'arbres binaires

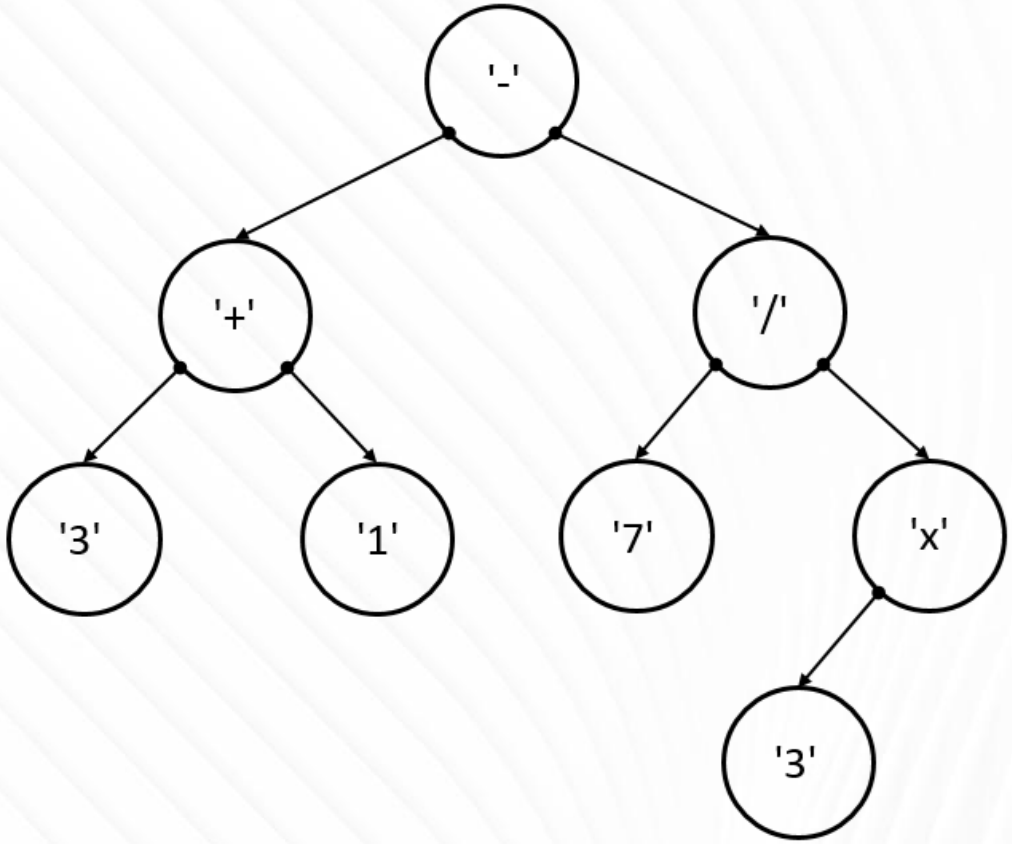
- Un arbre "strict" ou "localement complet" est un arbre où chaque nœud a soit 0 soit 2 fils.
- Un arbre binaire **complet** est un arbre dans lequel tous les niveaux sont remplis à l'exception **éventuelle** du dernier, dans lequel les feuilles sont alignées à gauche.
- Un arbre "parfait" est un arbre dont tous les niveaux sont complets.
- Un arbre "dégénéré" est un arbre dont chaque nœud a au plus un enfant (en fait, il s'agit d'une liste).

Illustrations



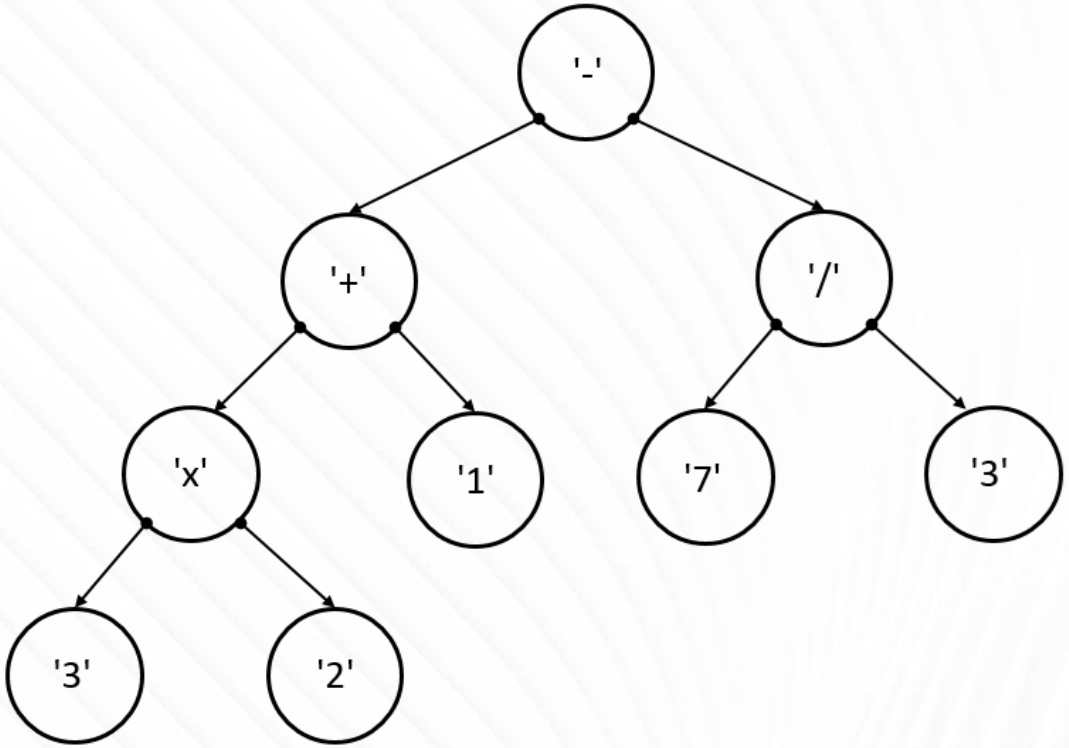
Strict	oui
Complet	non
Parfait	non

Illustrations



Strict	non
Complet	non
Parfait	non

Illustrations



Strict	oui
Complet	oui
Parfait	non

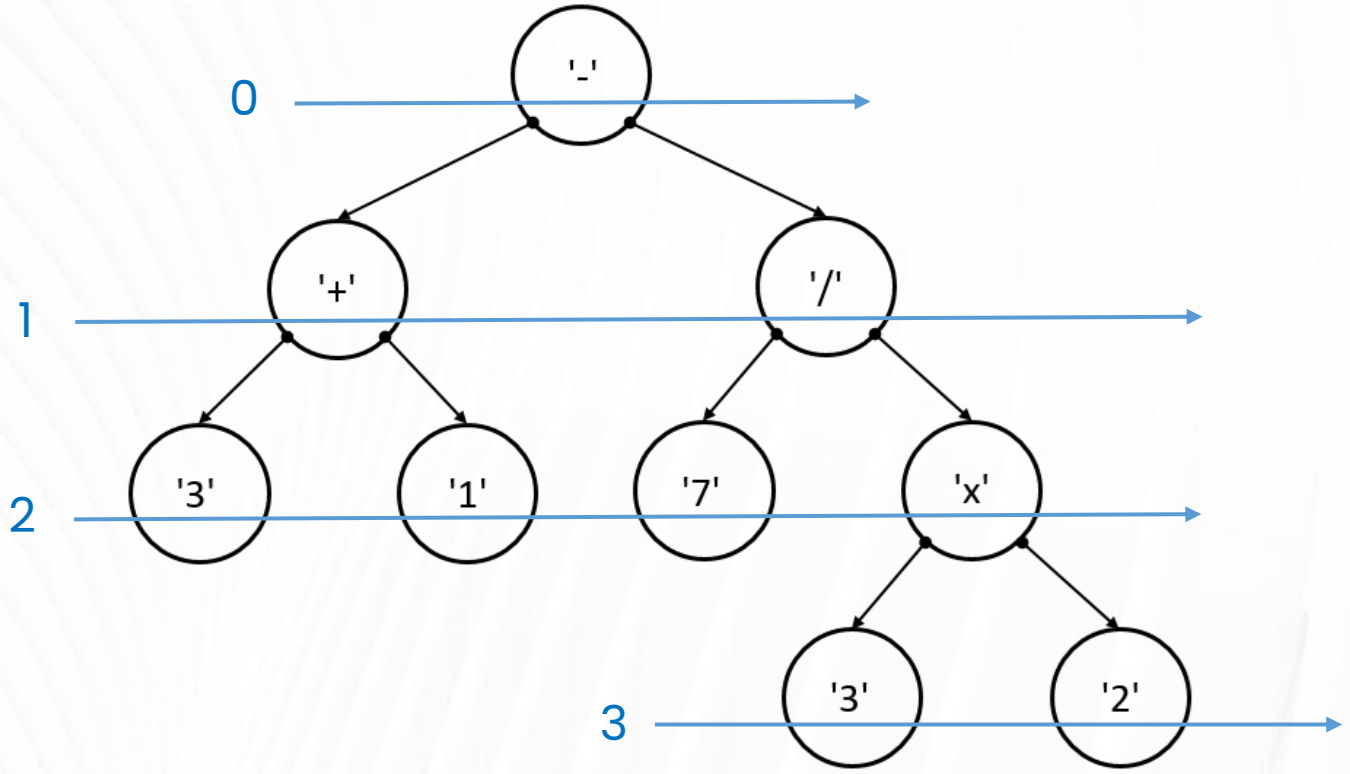
Illustrations

```
                                [root@]
                                   |
                                   v
                                 [001]
                 [L@]------------------+-----------------[R@]
                   |                                       |
                   v                                       v
                 [002]                                   [003]
       [L@]--------+-------[R@]                 [L@]--------+-------[R@]
         |                   |                     |                   |
         v                   v                     v                   v
       [004]               [005]                 [006]               [007]
  [L@]---+--[R@]      [L@]---+--[R@]        [L@]---+--[R@]      [L@]---+--[R@]
    |         |         |         |           |         |         |         |
    v         v         v         v           v         v         v         v
  NULL      NULL      NULL      NULL        NULL      NULL      NULL      NULL
```

Strict	oui
Complet	oui
Parfait	oui

Parcours en largeur

Il s'agit de visiter les nœuds d'un arbre, de la racine au dernier niveau, de gauche à droite à chaque niveau de profondeur.



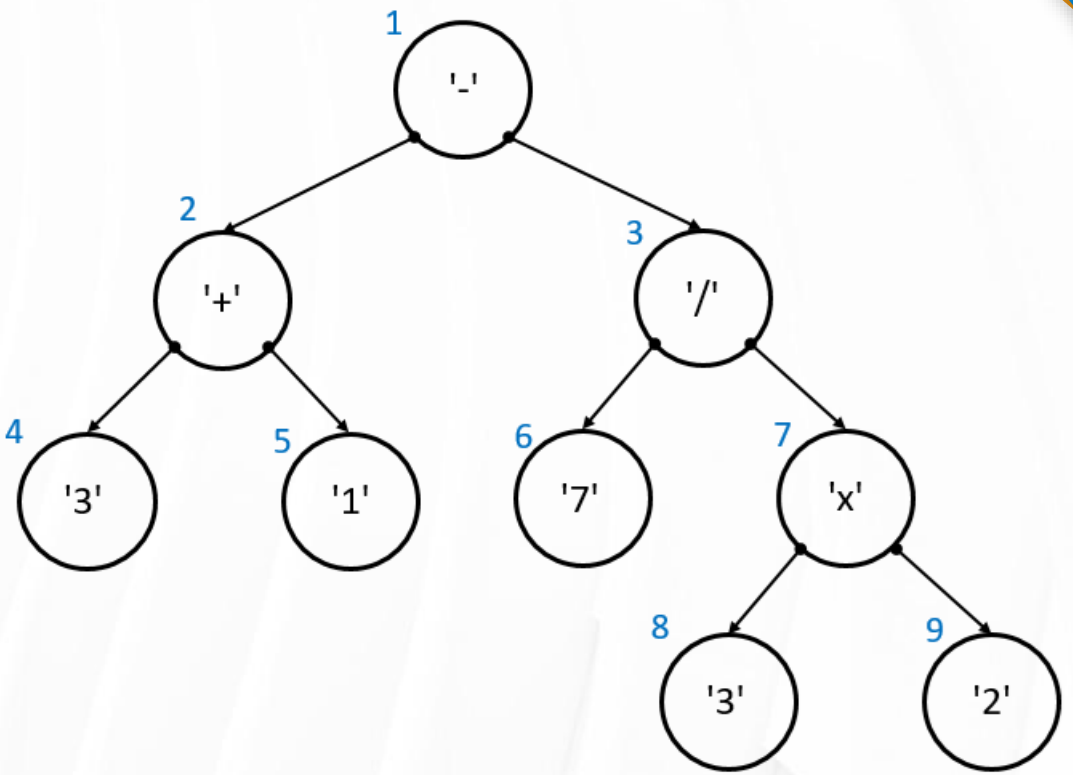
Sorties
- + / 3 1 7 x 3 2

Parcours en largeur

- Difficile de faire une définition récursive d’un parcours en largeur
- Pour chaque niveau, stockez les enfants des nœuds pour les visiter "après".
- Nous avons besoin d'une structure de données appropriée pour le faire…
- Numérotons les nœuds pour voir si nous pouvons trouver la structure de données qui convient à nos besoins…

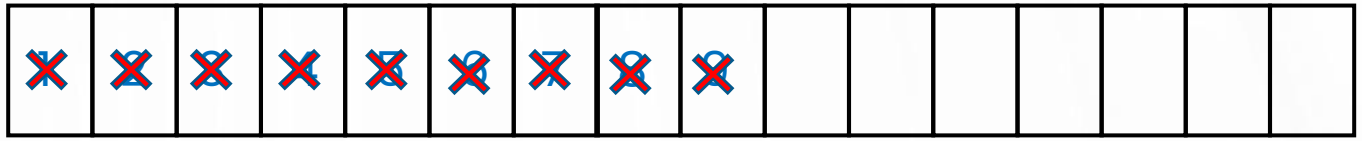
Parcours en largeur

Toujours commencer par 1 (le nœud racine) :
De 1 : stocker 2 et 3
<u>Niveau suivant</u>
De 2 : stocker 4 et 5
De 3 : stocker 6 et 7
<u>Niveau suivant</u>
De 4 : ne rien stocker
De 5 : ne rien stocker
De 6 : ne rien stocker
De 7 : stocker 8 et 9
<u>Niveau suivant :</u>
De 8 : ne rien stocker
De 9 : ne rien stocker
<u>Fin</u>



Comment les nœuds sont stockés ?

Toujours commencer par 1 (le nœud racine) :
De 1 : stocker 2 et 3
<u>Niveau suivant</u>
De 2 : stocker 4 et 5
De 3 : stocker 6 et 7
<u>Niveau suivant</u>
De 4 : ne rien stocker
De 5 : ne rien stocker
De 6 : ne rien stocker
De 7 : stocker 8 et 9
<u>Niveau suivant :</u>
De 8 : ne rien stocker
De 9 : ne rien stocker
<u>Fin</u>



Ce comportement devrait vous rappeler une structure de données que vous connaissez

C'est une file, stockant des valeurs de type `t_node *`

Parcours en largeur

Début de la fonction de parcours en largeur (à implémenter en TP)

```
void parcoursEnLargeur(t_tree t)
{
    t_queue q;
    t_node *cur;

    q = emptyQueue();
    enqueue(&q, t.root);

    while (isEmptyQueue(q) == 0)
    {
        cur = dequeue(&q);
       // traiter cur
       // mettre en file d'attente les enfants de cur s'ils existent
```

Arbres binaires de recherche (BST)

Soit l'arbre suivant :

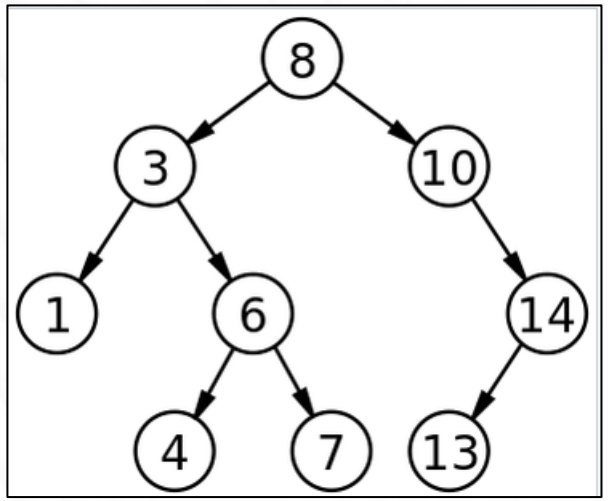


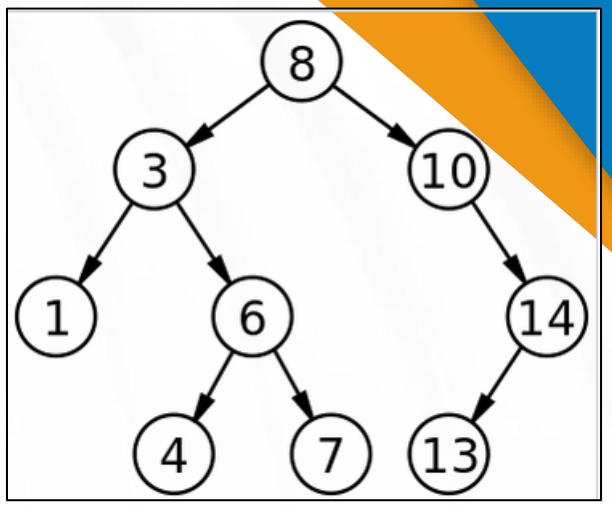
Image :
https://en.wikipedia.org/wiki/Binary_search_tree

Pouvez-vous noter une particularité des valeurs stockées dans les nœuds ?

Depuis n'importe quel nœud :

- Les valeurs stockées dans son sous-arbre de gauche sont inférieures à sa valeur.
- Les valeurs stockées dans son sous-arbre de droite sont supérieures à sa valeur

Arbres de recherche binaires (BST)



Exercice en direct : quels sont les résultats de :

Parcours préfixe?

Sorties
8 3 1 6 4 7 10 14 13

Parcours postfixe?

Sorties
1 4 7 6 3 13 14 10 8

Parcours infixe ?

Sorties
1 3 4 6 7 8 10 13 14

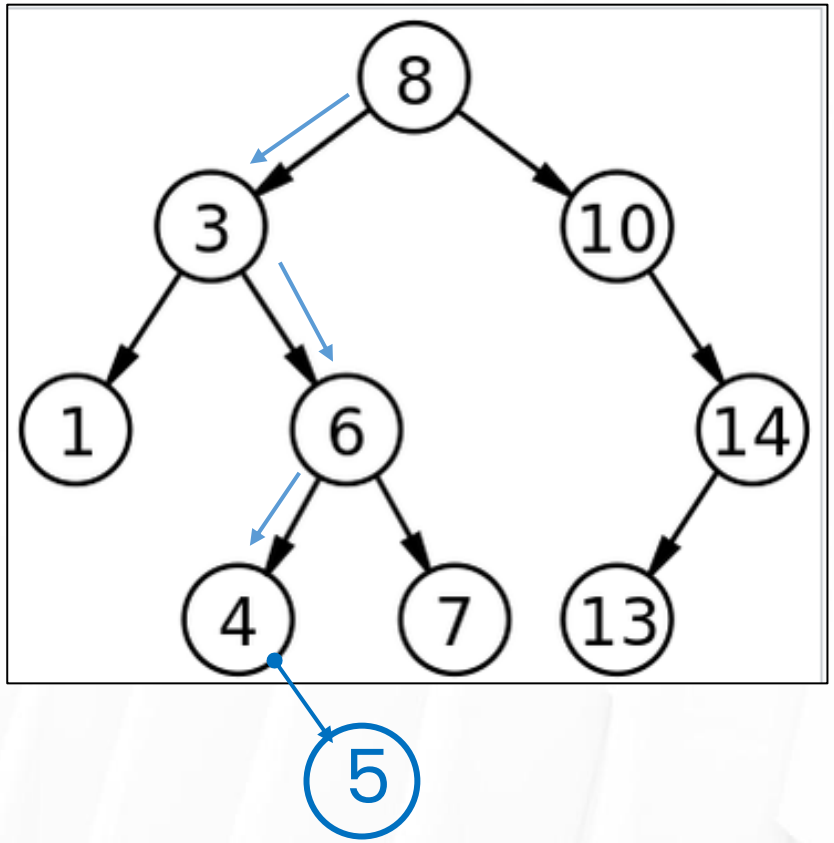
Insertion d'une nouvelle valeur dans une BST

Contexte : nous voulons insérer une valeur qui n'est pas encore présente dans l'arbre.

Insertion toujours en tant que feuille

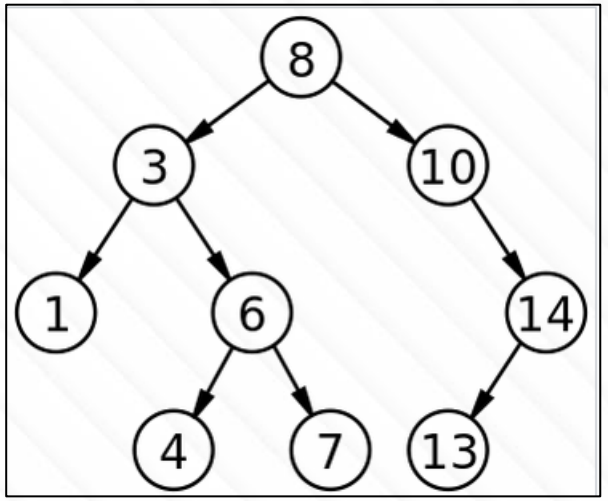
Exemple: insertion de 5

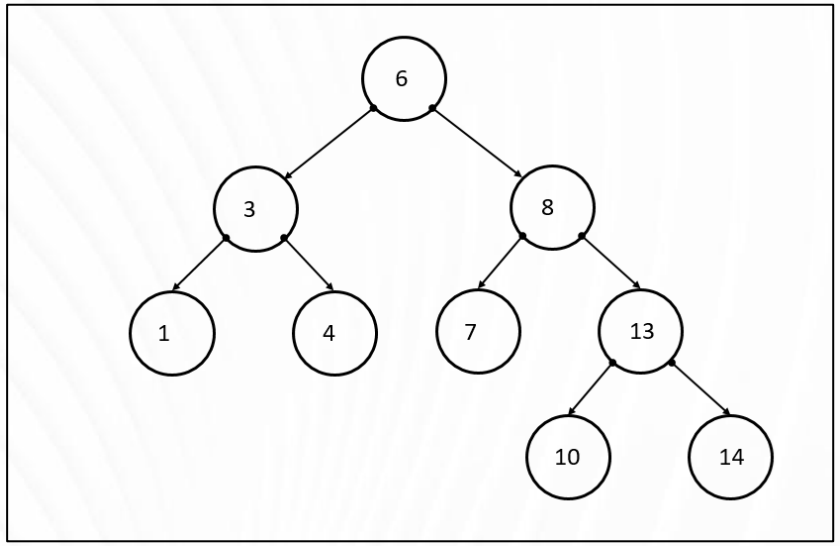
De 8 : « aller à gauche » (5 < 8)
De 3 : « aller à droite » (5 > 3)
De 6 : « aller à gauche » (5<6)
De 4 : « aller à droite » = insérer

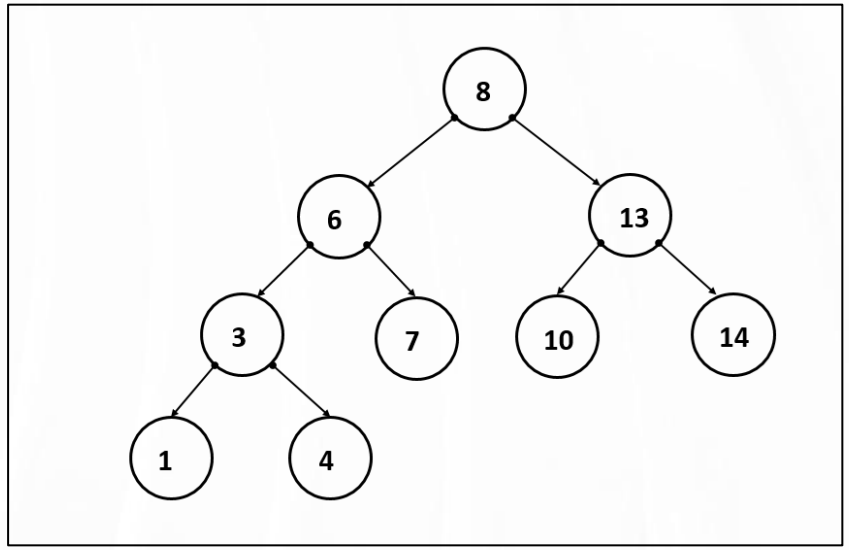


BST vs ordre d'insertion / équilibrage d'un arbre

Plusieurs BST peuvent stocker les mêmes valeurs !







Algorithme d'insertion de la BST

Soit `pn` un `t_node *` stockant la valeur (int) à insérer dans un arbre `mytree`

Si l'arbre `mytree` est vide, alors `pn` devient sa racine

Sinon, il faut parcourir l'arbre, en utilisant un `t_node *temp` et un `t_node *parent` (équivalent à `'prev'` pour les listes).

```
temp = mytree.root;
while(temp != NULL)
{
      parent = temp; // pour 'attacher' pn à la fin.
      aller à gauche ou à droite par rapport aux valeurs de pn et temp
}
```

pn devient l'enfant gauche ou droit de parent, par rapport aux valeurs de pn et de parent.

Exemples de programmes

```c
int main()
{
    t_tree mytree = createEmptyTree();
    insertBST(&mytree,8);
    insertBST(&mytree,6);
    insertBST(&mytree,10);
    insertBST(&mytree,14);
    insertBST(&mytree,7);
    insertBST(&mytree,4);

    displayTree(mytree);

    return 0;
}
```

```
                                [root@]
                                   |
                                   v
                                [008]
              [L@]------------------+-------[R@]
                |                             |
                v                             v
              [006]                         [010]
     [L@]--------+-------[R@]          [L@]---+-------[R@]
       |                   |              |              |
       v                   v              v              v
     [004]               [007]          NULL           [014]
[L@]---+--[R@]      [L@]---+--[R@]                [L@]---+--[R@]
  |         |         |         |                   |         |
  v         v         v         v                   v         v
NULL      NULL      NULL      NULL                NULL      NULL
```

Mêmes valeurs mais

```c
int main()
{
    t_tree mytree = createEmptyTree();
    insertBST(&mytree,6);
    insertBST(&mytree,7);
    insertBST(&mytree,10);
    insertBST(&mytree,14);
    insertBST(&mytree,8);
    insertBST(&mytree,4);

    displayTree(mytree);

    return 0;
}
```

```
                [root@]
                   |
                   v
                 [006]
      [L@]--------+-------[R@]
        |                   |
        v                   v
      [004]               [007]
  [L@]---+--[R@]      [L@]---+-----------------[R@]
    |         |         |                       |
    v         v         v                       v
  NULL      NULL      NULL                    [010]
                                        [L@]--------+-------[R@]
                                          |                   |
                                          v                   v
                                        [008]               [014]
                                   [L@]---+--[R@]      [L@]---+--[R@]
                                     |         |         |         |
                                     v         v         v         v
                                   NULL      NULL      NULL      NULL
```

Meilleur cas, en insérant 9 (à partir du premier exemple)

```c
int main()
{
    t_tree mytree = createEmptyTree();
    insertBST(&mytree,8);
    insertBST(&mytree,6);
    insertBST(&mytree,10);
    insertBST(&mytree,14);
    insertBST(&mytree,7);
    insertBST(&mytree,4);
    insertBST(&mytree,9);

    displayTree(mytree);

    return 0;
}
```

```
                                [root@]
                                   |
                                   v
                                [008]
              [L@]------------------+-----------------[R@]
                |                                       |
                v                                       v
              [006]                                   [010]
     [L@]--------+-------[R@]                [L@]--------+-------[R@]
       |                   |                    |                   |
       v                   v                    v                   v
     [004]               [007]                [009]               [014]
[L@]---+--[R@]      [L@]---+--[R@]      [L@]---+--[R@]      [L@]---+--[R@]
  |         |         |         |         |         |         |         |
  v         v         v         v         v         v         v         v
NULL      NULL      NULL      NULL      NULL      NULL      NULL      NULL
```

Le pire des cas...

```c
int main()
{
    t_tree mytree = createEmptyTree();
    insertBST(&mytree,4);
    insertBST(&mytree,6);
    insertBST(&mytree,7);
    insertBST(&mytree,8);
    insertBST(&mytree,9);
    insertBST(&mytree,10);
    insertBST(&mytree,14);

    displayTree(mytree);

    return 0;
}
```

```
[root@]
    |
    v
  [004]
[L@]---+-------[R@]
  |              |
  v              v
NULL           [006]
          [L@]---+-------[R@]
            |              |
            v              v
          NULL           [007]
                    [L@]---+-------[R@]
                      |              |
                      v              v
                    NULL           [008]
                              [L@]---+-------[R@]
                                |              |
                                v              v
                              NULL           [009]
                                        [L@]---+-------[R@]
                                          |              |
                                          v              v
                                        NULL           [010]
                                                  [L@]---+-------[R@]
                                                    |              |
                                                    v              v
                                                  NULL           [014]
                                                            [L@]---+--[R@]
                                                              |         |
                                                              v         v
                                                            NULL      NULL
```

Ceci est une liste, où 'right' est équivalent à 'next'.

Problèmes de complexité des BST

La structure de données BST est adaptée à une recherche efficace :

- Il peut utiliser une méthode dichotomique pour trouver une valeur avec une faible complexité si l'arbre est "équilibré".
- Le meilleur cas est un arbre « parfait » ou un arbre « complet ».
- Le pire cas est un arbre « dégénéré » (c'est-à-dire une liste)